

AI Infrastructure in Space: How Far Can We Go?

Qing Li^{*†}, Qiyang Zhang[‡], Daliang Xu^{*}, Tianze Huang[‡], Dingge Zhang^{*}, Yihao Zhao[‡], Xiaolong Huang[‡], Jinfeng Wen^{*}, Xiameng Hu^{*}, Tao Qi^{*}, Mengwei Xu^{*}, Shangguang Wang^{*}, Xuanzhe Liu[‡]

^{*}Beijing University of Posts and Telecommunications

[‡]Peking University

[†]Corresponding author: Qing Li (qingli@bupt.edu.cn), China

Abstract

Satellites are becoming programmable computing platforms capable of running increasingly demanding AI workloads. This shift raises a systems problem: how can AI services remain deployable, manageable, and recoverable after launch when compute capacity, connectivity, energy, and thermal headroom vary over orbital time? This paper develops a systems vision for *AI infrastructure in space*. We define it as the systems layer that manages AI capabilities across spacecraft, orbital networks, ground stations, and cloud backends, while treating orbital and physical state as part of the resource model. We synthesize relevant foundations from terrestrial AI infrastructure, satellite networking, and satellite edge computing, and examine the physical constraints that directly shape system design. We further ground this vision in three in-orbit case studies spanning the node, platform, and service levels. Telemetry from BUPT-1 satellite shows that usable compute capacity is bounded by thermal and energy envelopes. *SateLight* on BUPT-2 satellite reduces application-update transmission latency by 56.54% on average and up to 91.18%, with 100% update correctness. A stateful VLM serving case further shows that thermal interruptions make execution-state recovery a first-class systems problem. These observations motivate a research agenda for space-native resource management, lifecycle support, and sustained AI service across space and ground.

1 Introduction

AI progress increasingly depends not only on model architectures, but also on the infrastructure that makes models usable at scale. Modern AI systems require more than accelerators: they depend on memory and storage systems, high-speed interconnects, schedulers, deployment pipelines, and operational tools for serving, updating, and recovering models [1–5]. On Earth, these capabilities have been consolidated in datacenters, where AI infrastructure is built around grid power, active cooling, dense network fabrics, and human maintenance [6–9]. As AI systems scale, however, this datacenter-centric model faces increasing pressure from power delivery, cooling capacity, land and water use, geographic latency, and the cost of moving data from its source to centralized compute.

Meanwhile, advances in launch systems, solar power, onboard accelerators, optical links, and programmable spacecraft are making increasingly capable computing systems possible in orbit. SpaceX has outlined large constellations of solar-powered orbital datacenters, Google’s Project Suncatcher explores scalable machine-learning systems in space, and Starcloud is developing GPU-equipped orbital computing platforms. At the same time, AI has already been deployed onboard satellites for tasks such as cloud detection, telemetry analysis, link optimization, and autonomous planning [10–12].

As these capabilities become more general and programmable, the systems problem extends beyond individual applications to the infrastructure that keeps them operational after launch. Models and their associated state must be deployed, updated, scheduled, monitored, and recovered across spacecraft, orbital links, ground stations, and cloud backends [5, 13, 14]. We refer to this systems layer as *AI infrastructure in space*.

Building this infrastructure requires a different resource model from terrestrial AI systems. Datacenter schedulers typically operate on a relatively stable pool of accelerators, supported by continuous power, active cooling, and high-speed networks [3, 6, 7, 15]. In orbit, usable compute capacity changes over time. Solar exposure and battery state determine the available energy budget, while limited heat rejection constrains how long intensive computation can be sustained. Connectivity follows orbital motion and contact opportunities rather than remaining continuously available. Satellites also differ in their energy and thermal states, network visibility, mission roles, and fault conditions, making them difficult to treat as interchangeable compute nodes. Because hardware cannot normally be repaired after launch, resource management must also account for reliability and recovery [16–21]. These properties make orbital compute a time-varying and non-fungible resource rather than a fixed pool of interchangeable accelerators.

A space-native AI infrastructure must turn this physical and orbital state into actionable information for system software. Run-times should use platform telemetry and forecasts to guide workload placement, admission control, and service continuity [11, 16, 22]. Orbital networks must support the movement of models, data, and execution state across satellites and ground systems, while the software stack must manage deployment, update, rollback, and recovery after launch [5, 13, 14]. The objective is sustained and recoverable AI service over time rather than nominal compute capacity alone.

This paper presents a systems vision for AI infrastructure in space, grounded in both prior research and in-orbit experience. We first review the computing foundations most relevant to this problem, including terrestrial AI infrastructure, orbital networking, and satellite edge computing, and examine how their assumptions change in space. We then discuss the physical conditions that directly shape system design, with a focus on energy, thermal management, radiation, and spacecraft integration. We further ground the discussion in three in-orbit case studies that span the node, platform, and service levels. Measurements from BUPT-1 show that usable COTS computing capacity is bounded by time-varying thermal and energy conditions. *SateLight* on BUPT-2 demonstrates post-launch application update and rollback, reducing transmission latency by 56.54% on average and up to 91.18% while maintaining 100% update correctness. Finally, a stateful VLM serving case

shows how thermal interruptions turn long-running AI execution into a state-management and recovery problem. Based on these observations, we identify the key research challenges and outline a roadmap toward space-native AI infrastructure.

2 What Is AI Infrastructure in Space?

2.1 Concept and Scope

2.1.1 Definition: From AI for Space to AI Infrastructure in Space. We distinguish *AI for space* from *AI infrastructure in space*. AI for space uses models as mission tools, for example to detect clouds, flag telemetry anomalies, support autonomous planning or optimize links. AI infrastructure in space asks how such capabilities remain deployable, maintainable and reliable after launch. A spacecraft that runs one onboard model is AI-enabled. It becomes infrastructure only when that model can be deployed, updated, monitored, scheduled, recovered and coordinated with space and ground resources.

We define *AI infrastructure in space* as a cross-layer system for deploying, orchestrating, operating, updating, evaluating and reliably managing AI capabilities across spaceborne compute, orbital networks and space-ground systems under physical and operational constraints. It comprises five coupled layers: a *spaceborne compute layer* of processors, accelerators, memory and storage; an *orbital network layer* of inter-satellite links, space-ground links, gateways and contact windows; a *space-ground control plane* for deployment, monitoring, rollback, telemetry and scheduling; an *AI lifecycle layer* for model versions, weights, adapters, checkpoints, KV caches, intermediate features and data products; and a *physical-state interface* that exposes energy, thermal state, radiation risk, battery depth-of-discharge, attitude constraints, payload priority and mission safety margins.

The central question is whether AI can be operated as a managed service over time. Space AI infrastructure must therefore expose contact, power, heat, reliability and lifecycle state to the software stack, rather than burying them below it. Conceptually, the architecture forms a stack in which AI services sit above lifecycle and control-plane functions, with every layer coupled to communication constraints and the underlying physical state of the spacecraft.

2.1.2 System Boundary. The boundary of AI infrastructure in space is wider than the spacecraft. It includes onboard processors, accelerators, storage, runtimes and model-serving mechanisms, as well as the inter-satellite links, gateway access, ground-station schedules, model registries, update pipelines, mission-control software, cloud backends and digital twins that keep AI capabilities operational.

We draw this boundary around the lifecycle of an AI capability. Training or fine-tuning may occur on the ground; packaging and verification may occur in the cloud; upload is gated by contact windows; execution happens onboard; checkpoints and telemetry move across space and ground; and recovery may require rollback or fallback execution. Launch systems, propulsion, attitude control, orbital mechanics and payload hardware enter the model only when they affect AI operation. Orbit shapes contacts and energy cycles; attitude changes solar generation, antenna pointing and thermal exposure; payload priority constrains what AI may do. The transition is therefore from *running AI in space to operating AI as a managed space-ground service*.

2.1.3 Key Difference from Terrestrial AI Infrastructure. Terrestrial AI infrastructure is the closest design reference. Modern systems already rely on accelerators, high-speed datacenter fabrics, cluster managers, containerized deployment, model registries and serving runtimes [1, 6, 23, 24]. These mechanisms remain useful, but their assumptions change in orbit.

First, the compute node is not a stable server in a facility-managed pool. It is a spacecraft-bound resource whose usable capacity varies with solar exposure, eclipse, battery depth-of-discharge, thermal history, attitude, payload activity and mission rules. Onboard compute must therefore be modeled as physical-state-dependent capacity, not only as nominal TOPS or FLOPS.

Second, the network is not a persistent, low-latency datacenter fabric for collectives, storage access and rapid state movement [3, 15]. Satellites move; inter-satellite and space-ground links change; gateway access is limited by contact windows. The relevant resource is not only bandwidth, but when, for how long and with what reliability data can move. Delay-tolerant networking and contact-graph routing provide foundations [25], but AI infrastructure must also distinguish weights, checkpoints, KV caches, telemetry and sensor products.

Third, reliability cannot rely on physical maintenance. In orbit, radiation effects, software faults, link disruptions and thermal or power protection events require remote recovery, rollback, redundancy, checkpointing and degraded execution.

Finally, space nodes are not fungible. Satellites with identical processors may differ in orbit, contact opportunity, energy margin, thermal headroom, payload priority and fault risk. Model placement is therefore a joint compute, network, energy, thermal, reliability and mission-priority decision.

2.2 Real-World Attempts Toward AI Infrastructure in Space

Recent systems already point toward AI infrastructure in space, although none yet forms a complete stack. Together they demonstrate near-source inference, general-purpose spaceborne computing, programmable satellites, space-ground collaboration and early orbital data-centre visions. Their significance is the shift from fixed-function spacecraft toward programmable compute, network and service nodes.

2.2.1 Onboard AI Applications. Onboard AI is the most direct step toward near-source intelligence. Instead of downlinking all raw observations, a satellite can process data near the sensor and decide what to store, compress or transmit. ESA's Φ sat-1, for example, used onboard AI to filter low-quality or cloud-covered Earth-observation images before downlink [10, 26]. This reduced data volume and relieved scarce bandwidth.

Such systems remain closer to AI-enabled missions than to infrastructure: they typically bind one task, model and platform. The missing layer is lifetime update, monitoring, scheduling, recovery and coordination across space and ground.

2.2.2 Spaceborne Computing Platforms. Spaceborne computing platforms address the execution substrate. HPE's Spaceborne Computer and Spaceborne Computer-2, developed with NASA and deployed on the ISS, tested commercial high-performance computing

in space and supported edge-computing experiments [27, 28]. They show that space systems can host general-purpose computing, not only fixed avionics or payload processors.

For AI infrastructure, this is necessary but insufficient. A platform becomes infrastructure only when its resources are managed as part of a service stack. Schedulers must see not only processor type and storage capacity, but also power, thermal headroom, contacts, reliability risk and mission priority.

2.2.3 Cloud-Native and Programmable Satellites. Programmable satellites shift the focus from hardware capability to operational flexibility. Containerized deployment, isolation, remote update, rollback, telemetry and ground-orbit DevOps make satellites behave less like fixed payloads and more like maintainable platforms. This matters for AI because models, adapters, preprocessing logic and serving runtimes evolve after launch.

Frameworks such as SateLight illustrate this direction by making application delivery, differential upload, onboard reconstruction and rollback first-class satellite functions [14]. The implication is simple: evolving AI services require safe, observable software change.

2.2.4 Space-Ground Collaborative AI Systems. Near-term space AI infrastructure will be hybrid. Satellites are constrained by power, cooling, storage and maintainability; ground systems provide abundant compute and mature software ecosystems. Practical architectures therefore place latency-sensitive and bandwidth-saving tasks onboard, while training, validation, aggregation, long-term storage and fallback execution remain on the ground.

Cloud-integrated ground services already point in this direction. Microsoft Azure Orbital, for example, presents the ground segment as a cloud-connected platform for satellite communication, control, data processing and scalable operations [29]. Such systems are not space AI infrastructure by themselves, but they provide the ground-side fabric for model deployment, telemetry, data products and recovery. Space-ground collaboration is therefore the default architecture, not an implementation detail.

2.2.5 Orbital Data Centers and Space AI Service Visions. A more ambitious direction is the orbital data centre. Axiom Space has described orbital data-centre nodes for space-based storage, processing and AI/ML services [30]. Recent reports describe SpaceX plans for early orbital AI computing demonstrations before broader commercial deployment [31]. These are visions or reported plans, not proof that hyperscale orbital AI datacentres are practical. Their importance is conceptual: they reframe space from a mission-specific environment into a possible substrate for selected compute, storage and AI services. Launch cost, power generation, thermal rejection, radiation tolerance, hardware replacement, ground egress, orbital safety and utilization remain open constraints. Even small hybrid systems make the infrastructure question unavoidable.

2.3 From Isolated Attempts to Infrastructure

The attempts reviewed above show a clear trend: satellites are moving from fixed-function mission assets toward programmable compute, network and service nodes. Onboard AI demonstrates near-source inference; spaceborne computing platforms validate

general-purpose processing; programmable satellites enable post-launch software evolution; space-ground services connect orbital assets with cloud systems; and orbital data-centre visions point to longer-term infrastructure [10, 14, 26–31].

2.3.1 What Has Been Demonstrated. Existing systems have demonstrated key pieces of the stack: onboard inference can reduce downlink demand; commercial compute can operate in space; software can be remotely deployed, updated, monitored and rolled back; and ground-cloud platforms can support communication, data processing and operations. AI capabilities can therefore exist in orbit and connect to ground workflows.

2.3.2 What Is Still Missing. What remains missing is the infrastructure layer that makes these capabilities reusable, schedulable and reliable across missions and platforms. Current systems are still tied to specific tasks, satellites, models or demonstrations. They lack common resource abstractions, model and state lifecycle management, and cross-layer orchestration across compute, contact windows, energy, thermal state, reliability and mission priority.

2.3.3 Key Takeaway. Existing systems demonstrate pieces of the stack, but not yet a space-native infrastructure layer. The next step is to move from *running AI in space* to *operating AI over space-ground systems*: satellites, orbital links, ground stations, model artifacts, telemetry and physical constraints must be managed together.

3 Computing Foundations and Related Work

3.1 Terrestrial AI Infrastructure

We use terrestrial AI infrastructure as the baseline whose mechanisms space systems may inherit, and whose assumptions they must revise. We include work that defines datacenter-scale AI compute, governs shared heterogeneous workloads or distributes training and inference across accelerators. The critical assumptions are abundant power, active cooling, fungible nodes and an always-on lossless fabric. None survives unchanged in orbit.

3.1.1 AI Datacenter Architecture. Modern AI datacenters centre on GPUs and TPUs as the unit of compute: their datapaths, high-bandwidth memory and interconnect define per-node capability [2, 3, 32], which is procured and reported through benchmarks [4, 33]. Because training throughput is bounded by collective communication, the network becomes the computer; this has driven non-blocking fabrics, optically reconfigurable topologies [3, 8, 9], and GPU fabrics tuned for LLM traffic and fault tolerance [15, 34]. The cluster is the management unit, since warehouse-scale operation relies on cluster managers that pool and allocate machines as one computer [1, 6, 7, 23, 35]. Power and cooling are treated as facility services: terrestrial designs assume grid power, water or air cooling and microsecond-scale lossless interconnect, and power is managed through over-subscription rather than scarcity [36–38]. Finally, at frontier-scale operation, public reports of ten-thousand-accelerator clusters show that stragglers, failures and imbalance, not peak FLOPS alone, limit operation [39–42].

Implication: the terrestrial stack treats compute, network, power and cooling as abundant, homogeneous and continuously available. Section 4 revisits these assumptions under orbital physics.

3.1.2 Resource Management and Scheduling. GPU schedulers exploit the iterative structure of training to reduce completion time and raise utilization through time-slicing, migration and information-agnostic policies [43–46]. Later schedulers address heterogeneity and elasticity, co-adapting allocation with batch size or goodput and reasoning about heterogeneous accelerators and elastic jobs [47–53]. Fine-grained sharing lets multi-tenant clusters provide guarantees and GPU sharing through cell-based allocation, co-execution, fast context switching and memory sharing [54–58]. Scheduler design is also trace-grounded, calibrated against production GPU traces [40, 59, 60], while general-purpose frameworks support elastic execution [24].

Implication: these policies assume interchangeable, always-available accelerators that can be packed, preempted, and migrated at will. LEO nodes violate this assumption because energy and thermal state gate availability.

3.1.3 Distributed Training and Inference. Training parallelism scales beyond one device by composing data, tensor, pipeline and expert parallelism, either automatically or by hand tuning [61–71]. Sparsity and mixtures of experts use conditional computation to expand parameter count at near-constant FLOPs, but increase all-to-all communication and routing pressure [42, 72–74]. Because long jobs fail often, fault tolerance makes checkpoint/restart, in-memory recovery, redundant recovery and preemption-aware reconfiguration central concerns [75–80]. For inference and serving, generative serving introduced iteration-level batching, paged KV-cache memory, prefill/decode disaggregation, statistical multiplexing, and KV-centric or migration-based scheduling [5, 13, 81–89], building on model-serving systems that set latency-SLO baselines [90–92]. To sustain efficiency under constraint, offloading, speculative decoding and optimized engines stretch fixed hardware [93–95], while model reports document full training and serving stacks [41, 42, 96, 97].

Implication: both training and serving assume synchronized collectives over high-bandwidth, lossless and always-on interconnects among co-located accelerators. A contact-window-mediated orbital fabric cannot provide this property.

3.1.4 Assumptions That Break in Space. The terrestrial stack is effective, but four assumptions do not survive the move to orbit. First, abundant, steady power gives way to harvested, cyclic energy: datacenter scheduling treats power as oversubscribable [37, 38], whereas in orbit usable compute is gated by solar exposure, eclipse and battery depth-of-discharge (Sections 4.1 and 5.2). Second, active cooling gives way to radiative-only heat rejection, as air and water cooling [36] yield to conduction and radiation, so that thermal headroom, not raw FLOPs, limits sustained serving (Sections 4.2 and 5.4). Third, fungible nodes give way to non-fungible nodes: schedulers pack, preempt and migrate across interchangeable accelerators [43, 47, 86], but satellites differ in orbit, energy state and contact opportunity (Sections 4.3 and 4.4). Fourth, the lossless fabric gives way to scheduled links, since distributed training and serving assume high-bandwidth lossless collectives [5, 62, 65] while orbital interconnect is contact-window-mediated and energy-coupled (Section 3.2).

Inherited: accelerator abstractions, parallelism strategies, cluster schedulers and lifecycle/serving mechanisms. **Broken:** the resource model, the failure model and the connectivity model. *Takeaway:*

space AI infrastructure should reuse terrestrial mechanisms while replacing their assumptions with physically grounded, predictable-but-intermittent models (Sections 4–6). Table 1 summarizes the replacement.

3.2 Satellite and Orbital Networking

We treat orbital networking as the interconnect and I/O fabric of space AI infrastructure, not as a stand-alone communication service. The relevant work constrains where AI workloads can be placed, when nodes are reachable, how fast models and state can move, and whether that movement can be planned.

3.2.1 The Orbital Network Substrate. The substrate is layered by orbit regime. GEO, MEO and LEO trade altitude against latency, coverage and per-node resources. LEO mega-constellations, including Starlink, OneWeb, Kuiper, Guowang and Qianfan, are plausible near-term homes for spaceborne compute [98]. Within a constellation, satellites connect through radio-frequency or optical inter-satellite links, often in a *+Grid* motif of two intra-plane and two inter-plane neighbours. Optical links already reach the ~100 Gbps class [99]. Their system-visible properties are link establishment delay, outage probability and topology reconfiguration time.

The space-ground segment adds user links and feeder or gateway links. These are increasingly optical, weather-coupled, offered as ground-station-as-a-service and studied as shared distributed resources [100]. The resulting fabric is asymmetric. In-space mesh capacity scales with constellation size, whereas ground egress is capped by gateways, spectrum and contact time. Data, and increasingly computation, therefore accumulate in orbit [101].

Implication: the near-term fabric is globally distributed and mesh-rich, but egress-constrained, mechanically steered, energy-coupled and never fully active at once.

3.2.2 Predictable Topology Dynamics. The distinctive property of orbital networking is predictable motion. Connectivity varies continuously, but ephemerides make topology computable hours to days ahead, unlike static datacenter fabrics or stochastic terrestrial MANETs. Snapshot and virtual-topology models, virtual nodes [102, 103], contact graphs, contact-plan design and time-expanded graph optimization [104, 105] turn a moving constellation into analyzable graph sequences.

Topology is also a design variable. ISL motifs set latency and throughput envelopes [106, 107], and whether routing through space beats ground relay remains contested [108, 109]. Predictability is imperfect: operational constellations are actively managed and reconfigured [110], while user and gateway handovers create minute-scale path churn.

Implication: predictability is the main asset orbital networks have that terrestrial fabrics lack. Current AI runtimes, however, expose no interface to consume it, a gap we revisit in the runtime challenges.

3.2.3 Moving Bits Under Intermittency. Given a known but time-varying topology, routing must trade stability against optimality as links appear and vanish. Snapshot shortest-path, geographic and source-routing schemes address changing graphs. Under intermittency, however, movement must be planned. Delay/disruption-tolerant networking, store-carry-forward, the Bundle Protocol and

Table 1: Terrestrial AI infrastructure assumptions and space-native replacements.

Dimension	Terrestrial AI infrastructure	Space-native AI infrastructure
Compute node	Server or accelerator tray inside a repairable, facility-managed pool.	Spacecraft-bound compute resource whose usable capacity varies with orbit, battery, thermal state, payload priority, and fault risk.
Power and cooling	Grid-backed power and active air or liquid cooling treated mainly as facility constraints.	Harvested solar energy, eclipse, depth-of-discharge, conduction, and radiative heat rejection directly gate service time.
Network fabric	Always-on, low-latency, high-bandwidth fabric for collectives, storage, and migration.	Scheduled and intermittent contact graph with link opportunities that depend on orbit, pointing, energy, and gateway visibility.
Scheduler model	Pools of mostly fungible devices optimized for utilization, fairness, and SLOs.	Non-fungible orbital nodes scheduled over time-varying contact, energy, thermal, reliability, and mission-priority envelopes.
Lifecycle state	Models, containers, checkpoints, and serving state move through stable datacenter networks.	Weights, deltas, adapters, checkpoints, KV caches, and telemetry must be staged across constrained uplinks, on-board storage, and rollback paths.
Failure and recovery	Failed hardware can often be repaired, replaced, rebooted, or masked by nearby capacity.	Radiation, thermal protection, power events, and link loss require remote recovery, degraded modes, redundancy, and auditable rollback.
Evaluation	Benchmarks emphasize throughput, latency, utilization, and cost under stable infrastructure.	Benchmarks must include per-orbit useful work, energy and thermal margins, contact usage, recovery behavior, and mission safety.

Contact Graph Routing [21, 111, 112] therefore bear directly on transferring weights and checkpoints. Egress depends on contact-window and ground-station scheduling, which is now optimized for latency and throughput across distributed ground networks [113, 114], and is coupled to onboard data reduction.

At the transport layer, TCP and QUIC degrade over LEO paths because handovers inject loss and jitter. Link-layer-informed adaptation [115] and measurements of operational constellations [116, 117] matter because they bound how fast weights, checkpoints and state move within contact windows. This understanding depends on emulators and testbeds such as Hypatia, StarPerf, StarryNet, Celestial, OpenSN and LEOcraft [118–123]; RHONE [124] extends this methodology toward space computing networks.

Implication: this work explains where and when bits can flow, but treats traffic as opaque bytes. It does not distinguish weights, KV caches, gradients and telemetry, nor co-schedule them with compute, energy or thermal state.

3.2.4 From Communication to Computing Interconnect. Taken together, four assumptions of terrestrial AI fabrics break in orbit. Static topology becomes predictable dynamics, so contact windows should be first-class scheduling resources. Microsecond, lossless RDMA becomes millisecond-to-second intermittent paths, so collectives require redesign rather than porting. Commodity bandwidth becomes a scheduled, energy-coupled resource because link activation competes with payload and thermal budgets. Any-to-any reachability becomes contact-window-mediated reachability. Existing work supplies the lower layer: topology and contact formalisms, dynamic-graph routing, DTN scheduling, downlink scheduling and emulation tools. The missing layer is AI-specific: traffic classes for model dissemination, state synchronization, checkpointing and telemetry; deadline- and contact-aware bulk transfer; predicted network state exposed to AI runtimes; and joint compute–network scheduling.

Takeaway: the orbital network should be recast as the interconnect and I/O layer of a space-native computing system. The contact window, not the link, is the basic allocation unit.

3.3 Satellite Edge Computing and Onboard Intelligence

Satellite edge computing moves computation toward the data rather than the data toward computation. Instead of downlinking raw observations for processing on the ground, satellites can run inference in orbit and return only compact, higher-value products. This shift raises a set of coupled questions—what to compute onboard, how to divide workloads across onboard processors, ground-station edge nodes and terrestrial clouds, how satellites should cooperate, and what runtime and evaluation support such systems demand—that we examine in turn below.

3.3.1 Data Filtering, Compression, and Selective Downlink. Modern Earth observation missions face a data deluge: sensing capability grows faster than intermittent space–ground links. This is acute for high-resolution optical, hyperspectral, SAR and video missions, where raw observations can exceed contact-window capacity. On-board processing moves the first stage of selection into orbit, allowing satellites to discard low-value scenes, prioritize urgent events or produce compact semantic products before downlink. Φ -Sat-1 and CloudScout demonstrate neural-network cloud screening for hyperspectral imagery [10, 125]. OPS-SAT shows reconfigurable in-orbit image classification, clustering, cloud detection and telemetry anomaly detection [126–129]. HYPSON-1 extends this direction with in-orbit sea–land–cloud segmentation on real hyperspectral data [130–132]. Together, these systems turn satellites from passive data sources into active information filters.

This shift changes how data reduction should be evaluated. Compression is usually judged by reconstruction fidelity under a bit

budget. Onboard intelligence should instead be judged by mission utility under communication, compute, storage and energy constraints. Outputs may be cloud masks, object chips, segmentation maps, embeddings, confidence scores or event alerts. They reduce downlink volume, but introduce model-dependent decision risk. Selective downlink should therefore be evaluated by bandwidth savings, downstream accuracy, timeliness, uncertainty calibration, auditability and the cost of discarding observations that later prove important [22, 133].

3.3.2 Compute-Network Joint Scheduling and Resource Allocation. Satellite edge computing operates under strict limits on processing, memory, storage, energy, thermal dissipation and contact opportunities. Local inference can reduce downlink demand and latency, but it competes with sensing, storage and communication for spacecraft resources. Scheduling must therefore decide where a task runs, when its data move and whether intermediate results are stored, forwarded or discarded. This has motivated studies of computation offloading, resource allocation, service placement and computing-aware routing across satellite-terrestrial networks [134–137].

Existing formulations optimize task offloading, bandwidth, power, compute, routing or service caching. Representative work studies cooperative offloading, energy-aware placement, QoS-aware resource placement and end-to-end offloading with multidimensional allocation [138–141]. For AI-driven Earth observation, delay or energy alone is insufficient. The scheduler must distinguish raw observations, compressed products, semantic masks, embeddings, alerts and model updates because these objects differ in urgency, size, uncertainty and downstream value. Kodan and Serval highlight this semantic dimension: computation should maximize delivered information value under latency and resource constraints [22, 142].

3.3.3 Multi-Satellite Collaboration and Distributed Intelligence. Multi-satellite collaboration follows from the distributed nature of constellation sensing. One satellite has limited revisit frequency, partial coverage and constrained onboard resources. Collaboration can exchange compact features or alerts, partition inputs, split DNNs across satellites or space-ground nodes, or update models through federated learning without centralizing raw observations [143–147]. Unlike terrestrial edge systems, these mechanisms must handle intermittent links, non-IID observations, heterogeneous resources and stale model or feature updates.

The need grows as satellite AI moves toward larger models. Foundation models can improve retrieval, segmentation, change detection and multimodal reasoning, but full onboard deployment is limited by memory, energy and thermal constraints. A practical direction is distributed execution across constellations and the ground segment [148].

3.3.4 Runtime, Reliability, and Evaluation Infrastructure. Operational satellite edge AI needs runtime support beyond mission scripts. A reusable runtime should expose workload structure, data-product type, deadline, priority, resource demand, confidence and fallback behavior. Schedulers can then choose onboard execution, ground-ingress execution or cloud execution. Reliability is part of the same interface because onboard AI may discard data or trigger actions before ground verification. Operational systems therefore need versioning, rollback, health monitoring, watchdogs,

confidence-aware transmission and safe degradation under missed contacts, throttling, low energy or thermal constraints.

Evaluation infrastructure remains immature. In-orbit experiments are expensive, safety-constrained and hard to reproduce. Conventional simulators often miss spacecraft resource dynamics and AI workload behaviour. StarryNet emulates integrated space-terrestrial networks, and RHONE adds satellite-level power, thermal, orbit, network and computation dynamics [120, 124]. Satellite edge AI benchmarks should also report model latency, memory footprint, confidence distributions, intermediate data sizes, update cost, energy use and mission utility. HYPSON-1 sea-land-cloud imagery and OPSSAT-AD telemetry provide starting points [129, 131], but integrated benchmarks spanning sensor data, workload DAGs, orbital traces, contacts, hardware profiles and mission-risk metrics are still missing.

3.4 Synthesis: Building Blocks, but Not Yet Infrastructure

3.4.1 What Existing CS Work Provides. The computing and networking literature already provides reusable mechanisms. Terrestrial AI infrastructure contributes accelerator-centric abstractions, cluster scheduling, distributed training, model serving, checkpointing and deployment. Orbital networking contributes contact prediction, delay-tolerant transfer, routing over time-varying graphs, ground-station scheduling and emulation tools. Satellite edge computing contributes near-source filtering, selective downlink, collaborative inference and mission-aware data products. The pieces exist: compute can be shared, models and state can move, workloads can be scheduled and network dynamics can be predicted.

3.4.2 What Remains Fragmented. The problem is composition. AI systems assume stable datacenter power, cooling and connectivity. Orbital networking treats traffic mostly as bytes, not as weights, checkpoints, KV caches, features or telemetry. Satellite edge systems often optimize one mission pipeline rather than a reusable service layer. Existing work therefore lacks a resource model that jointly exposes contact windows, compute capacity, storage pressure, energy margin, thermal headroom, reliability risk and mission priority. Lifecycle mechanisms are also fragmented: update, rollback, checkpointing, evaluation and recovery are separate functions rather than one operating substrate.

3.4.3 Toward a Space-Native AI Infrastructure Stack. The next step is not to port a datacenter stack into orbit. It is to rebuild the infrastructure interface around predictable physical and network state. Spaceborne compute should be a time-varying service envelope, not a fixed accelerator pool. Orbital links should be the interconnect and I/O layer for model artifacts, state, telemetry and data products. The control plane should coordinate deployment, scheduling, update, monitoring, rollback and evaluation across space and ground. Energy, thermal state, radiation risk, contact forecasts and mission safety must be visible to the AI runtime. The next section explains why these constraints are the resource model, not implementation details.

4 Physical Foundations of Space AI Infrastructure

4.1 Energy as a First-Class Resource

Energy determines service continuity. Energy is not only a cost in orbit; it is a condition for service continuity. An AI node must power payloads, avionics, attitude control, storage, radios, heaters, processors, and accelerators while preserving battery and bus margins across sunlight and eclipse. The provisioning question is therefore not peak TOPS under laboratory power, but whether inference, planning, compression, or learning can be sustained over an orbital horizon without violating platform reserves. Orbital edge-computing work treats satellites as distributed computers whose usable compute depends on onboard resource envelopes, contacts, and mission timing [11, 22]. COTS satellite measurements and space-air-ground surveys likewise show that deployable AI is bounded by coupled compute, energy, communication, and environmental constraints [12, 149]. Capacity should therefore be reported in energy-feasible units, such as useful inferences per orbit, observations filtered per watt-hour, or latency under specified power reserves.

Orbit and attitude shape solar supply. Solar supply is predictable only after orbital geometry, eclipse timing, array orientation, temperature, degradation, conversion efficiency, and attitude plans are considered. Orbital period and beta angle set sunlight and eclipse duration, while imaging, antenna pointing, formation control, and momentum management can reduce incident power through cosine loss or self-shadowing. Power-system and panel-orientation studies show how generation, storage, and load profiles vary with these choices [16, 150]. Energy-harvesting systems provide the relevant abstraction: the runtime should exploit a forecastable energy-arrival process instead of assuming a stationary budget [151, 152]. Ephemerides, attitude timelines, payload schedules, and contacts should therefore become a time-varying power envelope that the AI scheduler can query.

Battery state bounds usable compute. Battery capacity is also not directly convertible into AI work. Admissible load depends on state of charge, depth of discharge, discharge rate, cell temperature, aging, imbalance, conversion loss, and mission safety margins. Battery-aware scheduling shows that workloads with the same energy demand can have different lifetimes when their discharge profiles differ [153, 154]. Cyber-physical power management and intermittent-system designs further show that admission control must preserve protected functions and recovery reserves under supply uncertainty [155, 156]. A satellite runtime should enforce state-of-charge limits, temperature derating, C-rate limits, aging uncertainty, and safe-mode reserves, scaling or rejecting AI work before battery health or bus voltage becomes critical.

AI accelerators create power peaks. Specialized accelerators improve energy efficiency through reduced precision, data reuse, sparse execution, and domain-specific dataflows [2, 157, 158]. Yet low energy per inference does not guarantee benign instantaneous power. Dense tensor phases, memory bursts, wake-up, regulator transients, and overlap with radio transmission can create peaks far above the average. Computational sprinting shows the value of briefly exceeding sustainable power or thermal levels, but the same tactic is risky when the accelerator shares a regulated spacecraft

bus with payloads, actuators, heaters, storage, and radios [159]. Qualification should therefore report peak power, ramp rate, rail demand, memory and I/O power, and behavior under caps, and system software should coordinate accelerator phases with DVFS, radio duty cycles, and payload operations.

The full pipeline shares one energy budget. An AI output is produced by a pipeline, not a processor alone. The spacecraft senses, moves and stores data, preprocesses, infers, compresses or prioritizes outputs, schedules downlink, and may change thermal-control behavior. Near-real-time Earth-observation systems show that on-board and ground processing should be chosen jointly, because in-orbit filtering reduces latency and downlink demand but consumes onboard energy [142]. Contact-aware transmission work shows that communication energy and latency depend on temporal and spatial opportunities [114]. Satellite-computing studies that include real-time, energy, and temperature constraints show that execution, communication, and thermal behavior cannot be optimized separately [160, 161]. The scheduler should maximize mission value per constrained joule across the full sensing-to-delivery path.

Runtime must be energy-aware. The systems requirement is an AI runtime that exposes energy state to admission, scheduling, and degradation. It should combine ephemeris-derived harvesting forecasts, attitude and payload schedules, battery health, rail-level telemetry, and model profiles for latency, accuracy, memory, average power, and peak power. Adaptive inference and constellation emulation show how accuracy, latency, and energy can be evaluated against orbital dynamics before launch [124, 162]. Intermittent inference and time-sensitive intermittent learning show how execution can be split into restartable units when harvested energy is unstable [163, 164]. The runtime should reserve non-preemptible energy for critical subsystems, choose model variants over one or more orbits, and degrade explicitly when margins shrink.

4.2 Thermal Management as the Hidden Bottleneck

Vacuum makes cooling radiative. Thermal management is hidden but often binding because near-vacuum removes the convective heat sink used by terrestrial servers. Heat from an AI die must conduct through the package, board, structural interfaces, and heat-transport hardware before being rejected by radiation. Space thermal control therefore relies on conduction paths, radiators, coatings, heat pipes, capillary devices, phase-change materials, and view factors rather than fans and ambient airflow [17, 165, 166]. Small-satellite design further shows that the body, payload placement, external surfaces, and orbital heat inputs form one coupled network [18]. Compute planning must therefore reason about heat generation and heat rejection, not only electrical power.

AI chips create local hotspots. High-performance AI processors can create local hotspots even when the average spacecraft temperature is safe. Tensor units, memory controllers, regulators, stacked memories, and I/O PHYs concentrate heat, affecting timing, leakage, aging, package stress, and nearby payloads. Dynamic thermal-management work established that microarchitectural activity must respond to temperature, not average power alone [167]. HotSpot and multicore thermal-management studies link architectural power maps to compact thermal models and show that

placement and migration can reduce hotspots [168–170]. AI accelerators should therefore be characterized by spatial and temporal power maps, and schedulers should know which kernels heat which physical regions.

Radiators and layout bound heat rejection. Heat rejection is constrained by radiator area and orientation, surface optical properties, interfaces, structural layout, and blockage by arrays, antennas, baffles, or payload apertures. Deployable radiator and panel studies show that increasing rejection capacity is a mechanical and geometric problem [171]. CubeSat and pico-satellite analyses show that limited surface area, orientation, material choice, and packaging dominate small-spacecraft temperatures [172, 173]. Radiator work further emphasizes integration with heat transport and structure [174]. Sustained AI compute therefore depends on processor placement, heat-path quality, radiator view, and conflicts with power generation or communication.

Orbits impose thermal cycles. The orbital thermal environment is periodic and history-dependent. Solar flux, Earth albedo, planetary infrared radiation, internal dissipation, and radiative cooling change over each orbit, with sharp transitions at eclipse entry and exit. Studies of small satellites and passive control show that these transitions create repeating temperature swings shaped by thermal capacitance, coatings, geometry, and internal power schedules [18, 175–177]. AI execution interacts with these cycles: an inference burst before eclipse may leave stored heat with little rejection opportunity, while the same burst during a favorable radiative period may be safe. Scheduling should therefore use both current temperature and forecasted heat-rejection opportunity.

Sustained compute is thermally bounded. Peak TOPS or FLOPS does not define deployable AI capacity when heat cannot be continuously removed. Dark-silicon results show that power and thermal limits can leave much silicon unavailable for simultaneous activation [178]. Work on Heat-and-Run, thermal herding, and multiobjective thermal control similarly shows that throughput depends on duty cycle, thermal capacitance, spatial heat distribution, and cooling constraints [179–181]. In orbit, the heat sink is finite, radiative, orientation-dependent, and shared. Useful metrics should therefore report sustained mission throughput under thermal margins, transient warm-up, steady-state temperature, cooldown time, and allowed duty cycle.

Runtime must enforce thermal safety. A thermal-aware runtime should act before safety limits are reached. Proactive temperature balancing and hierarchical thermal management show that future thermal states can be shaped by present scheduling and by coordination between local controls and global placement [182, 183]. Temperature-aware and real-time satellite-computing work connects this principle to orbital workloads by considering deadlines, energy, and temperature together [160, 184]. A space AI runtime should maintain a reduced-order thermal model calibrated by sensors, map kernels to heat profiles, forecast sunlight and eclipse transitions, and choose among placement, DVFS, batching, model simplification, deferral, and safe degradation.

4.3 Radiation, Reliability, and Non-Maintainability

Radiation causes faults and degradation. Space radiation affects AI infrastructure through transient faults and cumulative degradation. Single-event upsets, transients, and latchup can corrupt state or create destructive current paths, while total ionizing dose and displacement damage degrade devices over mission lifetime. Foundational radiation-effects work shows that susceptibility depends on particle type, energy, device geometry, process technology, shielding, and operating voltage [19, 20]. Soft-error research shows that hardware faults become system failures through architectural and software masking chains, so vulnerability is a stack property rather than a device property alone [185]. SRAM-FPGA studies further show that configuration state can dominate reconfigurable-platform vulnerability [186]. Space AI must therefore bound the probability that corrupted weights, activations, metadata, storage, buffers, or control decisions reach mission outputs.

COTS reliability is uncertain. COTS processors and accelerators are attractive because they provide modern process nodes, high arithmetic density, mature software, optimized compilers, and fast deployment. Their weakness is radiation behavior that is often uncertain, workload-dependent, and insufficiently characterized for mission assurance. Neural-network radiation studies show that learning workloads may tolerate some bit errors yet fail sharply when faults strike critical weights, activations, or control structures [187, 188]. Proton and neutron irradiation studies further show that error modes depend on architecture, memory hierarchy, precision, and mitigation [189, 190]. Candidate devices should therefore be tested with representative models, precisions, compiler settings, memory layouts, and power modes, and the runtime should expect reliability to drift with aging and exposure.

Rad-hard computing trades speed for assurance. Radiation-hardened processors, protected memories, ECC, lockstep execution, triple modular redundancy, hardened nonvolatile memory, and scrubbing reduce important failure classes, but often lag COTS devices in performance, efficiency, memory bandwidth, accelerator support, tools, and software ecosystems. TMR and lockstep architectures show how redundancy improves reliability, while LEON-class processors illustrate hardened embedded cores in space computing [191, 192]. Radiation-tolerant nonvolatile memory and heterogeneous modular redundancy extend this toolbox for storage, checkpointing, and mixed-assurance systems [193, 194]. The design question is not rad-hard versus COTS in general, but which functions require hardened assurance and which can use COTS acceleration with containment, redundancy, and validation.

Non-maintainability demands autonomous recovery. After launch, failed boards cannot normally be replaced, connectors re-seated, debuggers attached, or subsystems manually power-cycled, and contact gaps delay ground intervention. Recovery must therefore be onboard. Autonomous recovery work shows how systems can regain functionality after component failures, and microreboots show that fine-grained restart can recover software components without rebooting an entire system [195, 196]. Virtual-machine and process checkpointing capture, migrate, or restore execution state after faults [197, 198]. Satellite AI services should be decomposed

into restartable tasks with bounded side effects, persistent checkpoints, watchdogs, safe-mode fallbacks, and policies for partial mission completion.

Fault tolerance spans the full stack. A reliable AI service requires more than a reliable accelerator. Radiation or aging can affect weights, activations, compiler output, allocators, drivers, operating-system state, storage metadata, scheduler decisions, and packets. TensorFI exposes neural-network sensitivity through fault injection, and Thales demonstrates algorithm-aware error detection and correction for DNN accelerators [199, 200]. Hypervisor replication masks failures below the application, while delay-tolerant networking preserves delivery semantics across disrupted links [21, 201]. Space AI fault tolerance should therefore be layered: ECC and scrubbing for memory, replication or diversity for critical computations, checkpointing for progress, sanity checks for outputs, and disruption-tolerant protocols for data movement.

Runtime must be radiation-aware. A radiation-aware runtime should connect environmental exposure, error telemetry, workload criticality, and update trust. Secure update and remote attestation support trusted deployed software, while Ranger and selective hardening show that neural-network resilience can target vulnerable operations rather than harden everything uniformly [202–204]. CLEAR shows how low-cost recovery can support resilient execution under expected transient faults [205]. The runtime should adapt scrubbing rates, checkpoint intervals, model variants, replication, voting, and placement based on orbit-dependent risk and measured faults, and it should support secure post-launch updates for models, kernels, and drivers.

4.4 Materials, Structures, and Packaging

Terrestrial servers do not fit orbit. Space AI infrastructure cannot be built by moving terrestrial servers into orbit. Data-centre servers assume grid power, active cooling, replaceable parts, benign radiation, continuous networking, human maintenance, and a stationary mechanical environment. Satellite computing measurements show that COTS devices become constrained by power, temperature, software deployment, and platform integration in orbit [149]. Constellation systems such as EagleEye and Geoduck show that onboard compute must be co-designed with sensing opportunity, orbital motion, contacts, and mission coordination [206, 207]. Space–air–ground surveys likewise frame space nodes as constrained edge infrastructure in a heterogeneous network [12]. The baseline should therefore be spacecraft-integrated computing, with physical survival, sustained thermal throughput, containment, and validated operating envelopes treated as first-order goals.

Materials determine cooling and lifetime. Materials set heat spreading, radiation exposure, contamination risk, structural survival, and lifetime. Graphene- and graphite-based spreaders can reduce local gradients, but their value depends on interface resistance, anisotropy, thickness, manufacturability, cycling stability, and structural compatibility [208, 209]. Shielding reduces particle exposure but adds mass, can generate secondary radiation, and may alter thermal paths, so it must be optimized rather than maximized [210]. Low-outgassing materials are also essential because condensable volatiles can contaminate optics, arrays, coatings, and sensors [211]. Thermal interfaces, adhesives, coatings, shields, underfills,

boards, and panels should therefore be selected as one coupled materials stack.

Packaging defines the thermal path. Heat does not travel from an AI die directly to space; it crosses die attach, package lid, thermal interface, board copper, vias, connectors, fasteners, structural panels, heat straps or pipes, and finally a radiator. Thermal optimization of 3-D integrated circuits shows that placement and inter-layer heat paths shape package-scale hotspots [212]. Liquid-cooling and multiobjective thermal-control research show that cooling capacity, transport cost, and temperature uniformity must be co-optimized [181, 213]. At spacecraft scale, radiator panels and heat-transport hardware show that structural layout determines which external surfaces can reject heat [174]. Design tools should propagate workload power maps through package, board, and spacecraft thermal networks rather than collapse the system into a single junction-to-ambient value.

Launch loads constrain hardware form. Before an AI module computes, it must survive vibration, acoustic loading, quasi-static acceleration, sine vibration, and separation or pyrotechnic shock. Structural-validation work emphasizes correlating analysis with modal and environmental testing rather than relying on unvalidated finite-element predictions [214]. Euclid virtual-shaker analysis and flight-like additively manufactured satellite structures illustrate the coupled design–analysis–test loop required for unconventional architectures [215, 216]. Random-vibration experiments show that board response, component mass, package location, and solder-joint fatigue can dominate electronics failure risk [217]. These loads disfavor tall heat sinks, unsupported accelerator cards, socketed modules, long connectors, and airflow-dependent layouts, so mechanical and thermal design must be verified together.

SWaP-C limits compute scaling. Mass, volume, power, and cost prevent scaling by simply adding accelerators. Extra compute can require larger arrays, more battery capacity, stronger power conditioning, larger radiators, heat transport, structure, shielding, and qualification. Multidisciplinary small-satellite optimization shows that subsystem and operational variables are tightly coupled, while structural sizing treats mass, stiffness, and performance as competing objectives [218, 219]. Constellation sensing studies add orbit design, fleet size, geometry, and communication strategy to the trade space [206]. Efficient accelerators reduce compute, memory, and data-movement cost, but do not remove spacecraft-level trade-offs [158]. The objective is mission value over a Pareto frontier spanning throughput, accuracy, latency, energy reserve, thermal margin, mass, volume, reliability, cost, and software maturity.

Co-design must span chip to spacecraft. The physical foundation of space AI infrastructure requires chip–package–board–spacecraft co-design. AI workload traces should inform accelerator architecture, memory hierarchy, package power maps, board placement, power distribution, structural modes, radiation protection, radiator sizing, and orbital thermal boundaries. OpenM-DAO illustrates multidisciplinary analysis for coupled models, and spacecraft optimization shows that physical variables and operational schedules can be optimized together [218, 220]. Cool-3D and optimization-architecture surveys show how architecture, thermal modeling, modularity, convergence, and model ownership can be balanced during early design [221, 222]. The deliverable should be a validated operating envelope and runtime policy set, not only a

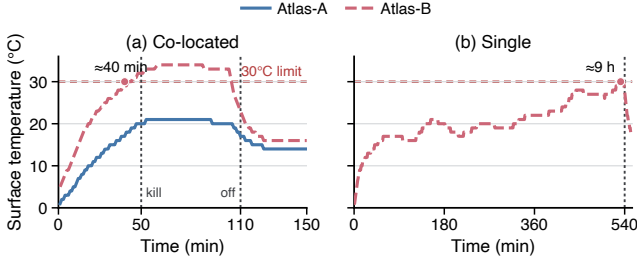


Figure 1: Thermal ceiling and the co-location effect: a shared radiating surface collapses a nine-hour single-device window to under two hours.

bill of materials; sustained, safe, recoverable AI service is the target, and silicon throughput is one input.

5 Case Studies: From Edge AI to Space-Native Infrastructure

Section 2.2 surveyed global efforts to deploy AI in orbit. Here, we trade breadth for depth by examining three first-party deployments on BUPT-1, a 12U Sun-synchronous satellite at approximately 490 km, and BUPT-2, a sibling platform with onboard cloud-native computing. Their telemetry, update logs, service-continuity records, and thermally matched 1:1 ground twin enable paired space-ground analysis rarely possible with external deployments.

The three cases form a dependency ladder from node to platform to service: Section 5.2 examines orbital computing capacity, Section 5.3 studies capability deployment and update, and Section 5.4 investigates stateful service continuity under interruption. Together, they reveal the physical, operational, and recovery abstractions required for dependable in-orbit AI systems.

5.1 The Node: Measuring the Physical Envelope of COTS Compute in Orbit

Our first case uses a satellite as an instrument for measuring what an orbital compute node *is*. On a 12U Sun-synchronous smallsat near 490 km, we flew two Raspberry Pi 4B boards and two Atlas 200 DK AI accelerators as COTS payloads, logging power, battery state, and structural surface temperature every few seconds over roughly four months. The question is not peak TOPS, but whether capacity can be treated as constant. It cannot. Usable capacity is modulated by two orbital constraints, a thermal ceiling and an energy budget, so the node is better modelled as a *physically modulated resource* than as a fixed server.

The thermal dimension is a hard ceiling enforced by the platform. Heat from a COTS chip conducts into a shared aluminium surface and, without convection, leaves only by radiation; $T_{\text{surf}}(t)$ then approaches the 30 °C structural limit. One accelerator at full load reaches this limit after about nine hours and stops cleanly. Two co-located accelerators share the same radiating surface: the surface reaches 30 °C in roughly forty minutes, the operating system kills the first task near fifty minutes, and the platform powers down the second device at about 110 minutes (Fig. 1). Co-location therefore collapses the safe continuous-compute window from about nine

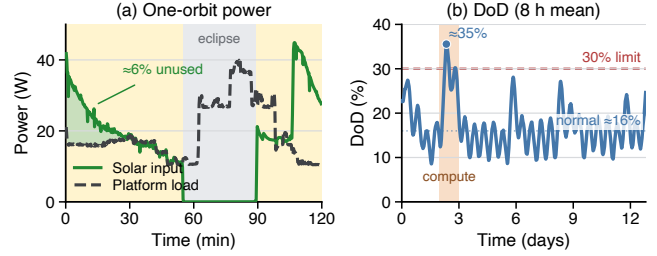


Figure 2: The energy budget and its two governors: solar power in sunlight, depth-of-discharge in eclipse.

hours to under two, showing that thermal capacity is shared across devices.

The energy dimension is one budget with two governors that switch at the terminator (Fig. 2). In sunlight, compute should track harvested solar power from below: exceeding it draws from the battery, while leaving too much converted solar power unused wastes energy and adds heat. About 6% of converted solar energy goes unused in our measurements. In eclipse, every joule comes from the battery, and the budget is the remaining depth-of-discharge (DoD) headroom. Solar is a perishable flow, whereas DoD is cumulative and history-dependent. Normal operation keeps daily-average DoD near 16%, but multi-day continuous computation drives it to about 35%, beyond the 30% design limit; raising the average from 25% to 30% costs on the order of a quarter of battery lifetime. Where a task runs in the orbit therefore changes both capacity and mission cost.

Both limits are clocked by orbital mechanics: a thermal ceiling driven by compute heat and unused solar, and an energy budget governed by solar input in sunlight and DoD in eclipse. In this half-year deployment, computation is bounded by thermal and energy constraints on minute-to-hour timescales; radiation is not the operative limit for this COTS node at this altitude and duration. Capacity also varies within a device, as thermal throttling lengthens inference latency by about 10% on some models, although this effect is secondary to platform-level task termination.

The missing piece is an abstraction, not telemetry. T_{surf} , DoD, and solar phase are already measured and largely predictable. Yet the compute layer still sees a flat server rather than a forecasted physical envelope. Platform state must be exposed as a schedulable resource, and long-running services must be built to survive involuntary physical preemption.

Implication: an orbital node is a physically modulated resource, with capacity metered by surface temperature, solar power, and depth-of-discharge. Like contact windows, this envelope is predictable, but it remains largely invisible to the scheduler that must plan against it.

5.2 The Platform: Application Delivery and Update as the Orbital Control Plane

To support onboard software maintenance, we propose *SateLight*, a container-based framework for satellite application updates. As shown in Figure 3, ground-side tooling compares original and updated containerized applications, extracts content-aware deltas,

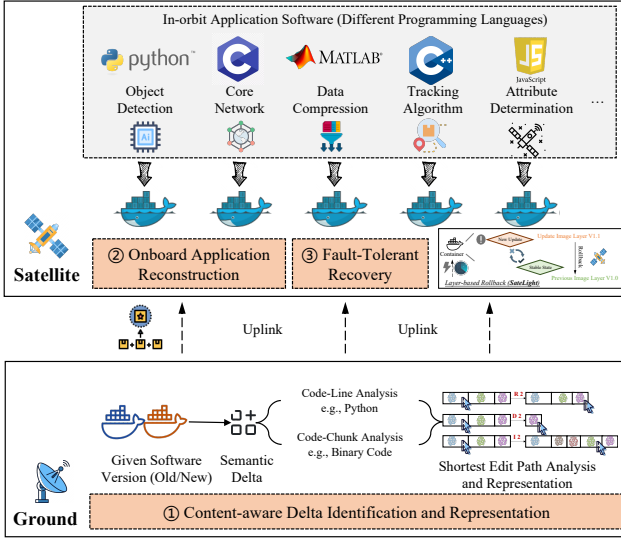


Figure 3: The overall workflow of SateLight.

and encodes semantic changes into compact metadata. The onboard component reconstructs the application through fine-grained updates and uses container layering for fast rollback after failure. We deploy *SateLight* on BUPT-2, an operational LEO satellite with cloud-native onboard computing and ground communication. Across heterogeneous applications, *SateLight* reduces ground-to-satellite transmission latency by up to 91.18%, with an average reduction of 56.54%, and achieves 100% update correctness.

- **Deployment and capability.** SateLight spans ground and satellite with containerized applications, content-aware differential upload, fine-grained onboard reconstruction, and container-layer rollback. It was exercised on ten applications across six languages, from CV inference to 5G core functions, and validated by a live in-orbit update on BUPT-2.
- **Infrastructure reading.** A satellite becomes a platform only when what runs in orbit can be changed safely after launch. Containerization handles application heterogeneity; semantic deltas fit kbps uplinks and ten-minute contacts; layer rollback provides recovery without waiting for the next ground pass.
- **Binding constraints and evidence.** Uplinks run at tens-to-hundreds of kbps with 4–6 ten-minute contacts per day. A full representative vision container costs ~ 10.7 hours of pure uplink time at 200 kbps, roughly two weeks of wall-clock contacts, whereas a content-aware delta for a 10% code change is tens of kilobytes and ships in seconds. Average transmission-latency reduction is 56.5% over the best baseline, up to 91%, with ~ 2 s onboard reconstruction overhead and 100% correctness. Failed updates roll back autonomously with 0.48 ms backup preparation and ~ 36 s recovery.
- **AI specificity.** Model weights and adapters are among the largest, most frequent artifacts AI infrastructure must ship,

and they are binaries. The chunking used for executables can also delta-ship future weight updates. Container layers provide version-level reversibility, while Case III supplies finer execution-state continuity.

- **Missing abstraction.** SateLight assumes stateless applications and one satellite. Kbps uplinks cannot carry GB-scale weight refreshes even as deltas, pointing to feeder-link capacity, on-orbit adaptation, fleet-scale staged rollout, and update scheduling constrained by energy, thermal state, and contacts.

Implication: operability precedes intelligence. Without delivery, update, and rollback, an onboard model is a one-shot demo; with them, the satellite becomes an iterable platform.

5.3 The Service: Sustaining Stateful VLM Inference Across Thermal Interruptions

Our third case asks whether a stateful AI service can make forward progress when its compute substrate is repeatedly removed by spacecraft protection mechanisms. We deploy *Rover*, an intermittent-aware VLM inference runtime, on BUPT-1 and BUPT-2 using COTS payloads. Sustained Qwen3-VL-2B inference raises device temperature from 45°C to the 55°C shutdown threshold in only 48 s, while a video-level Earth-observation request can require several minutes of continuous reasoning. A conventional runtime therefore loses visual features, prefill progress, KV caches, and partial outputs whenever the spacecraft bus powers off the payload, and repeated restart from scratch may prevent the request from ever completing.

Rover treats these power losses as recoverable execution boundaries rather than terminal failures. Its checkpoint policy follows the structure of the VLM pipeline. During vision encoding, it persists layer-boundary hidden states because bidirectional attention prevents patches from being resumed independently. During language-model prefill, it partitions execution jointly along token and layer dimensions and checkpoints only the KV tensors and boundary hidden states needed to reconstruct a two-dimensional progress frontier. During decoding, it batches multiple KV-cache updates into each checkpoint, avoiding the fragmented writes and high normal-path overhead of per-token persistence. Phase-specific layouts and committed checkpoint records further ensure that incomplete writes are not mistaken for valid recovery state.

Recovery is also scheduled around model dependencies rather than replayed in a fixed layer-first or chunk-first order. After reboot, Rover restores the weights, KV tensors, and hidden states that unlock useful computation earliest, while overlapping later SSD reads with dependency-ready blocks. This is important because arbitrary shutdowns leave an irregular staircase-shaped frontier: some token-layer blocks have completed, others must be recomputed, and each candidate recovery order exposes a different amount of storage I/O. Rover reduces this restoration problem to a lightweight scheduling plan that can be solved within 100 ms.

Experiments driven by in-orbit thermal traces show up to a $3.2\times$ end-to-end speedup over existing checkpointing baselines, up to an 85.6% reduction in recovery latency, and up to a $3.56\times$ reduction in SSD writes. Normal-path energy overhead remains between 3.7% and 10.4%. In the real-world satellite deployment, Rover completes most of the requests while the baseline completes none. Figure 4

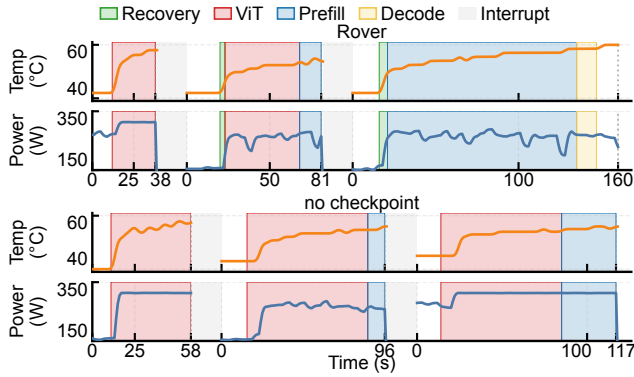


Figure 4: Thermal trace for a real-world deployment of Rover.

shows a representative trace where a request spans three execution windows and two payload resets, but resumes from committed progress and finishes in the third window.

Implication: for stateful orbital AI, survivability is an AI-state management problem. A long-running service exists only when its execution state is explicit, durable, and recoverable across physical interruptions.

5.4 Lessons Learned

- **The node is not a server.** Capacity is a physics-modulated time series over two measurable variables, enclosure temperature and depth-of-discharge. These envelopes must become scheduler inputs (Case I; Sections 6.1, 6.3).
- **At LEO, thermal and energy bind before radiation.** Multiple months of unhardened COTS operation produced zero observed SEE errors, while overheating terminated tasks twice. Near-term infrastructure effort should prioritize thermal and energy management (Case I; Section 4).
- **Operability precedes intelligence.** Without delivery, update, and rollback sized to kbps uplinks and minute-scale windows, each onboard model remains a one-shot demo. Lifecycle support is the entry ticket to infrastructure (Case II; Sections 6.4, 6.5).
- **Demanding services shift the bottleneck from compute to state.** Container layers provide version-level reversibility, and runtime checkpoints provide execution-level continuity. Both are state management under physics-driven preemption (Cases II–III; Section 6.4).
- **Predictability generalizes beyond topology.** Energy harvest and thermal accumulation, like contact windows, are forecastable from orbital mechanics. Infrastructure should schedule around forecasts rather than react to thresholds (Cases I and III; Sections 6.2, 6.3).
- **Hybrid-first, single-node so far.** Each working system is a space–ground loop, with the ground packaging, validating, and deciding while orbit executes. All three cases remain single-satellite; constellation-scale orchestration is still open (all cases; Section 6 and the roadmap).

Takeaway: in orbit, communication and computation are both metered by predictable physical cycles. Space-native AI infrastructure

is the systems layer that turns those forecasts into scheduling decisions.

6 Research Challenges: Toward Space-Native AI Infrastructure

The previous sections show that AI infrastructure in space is a stack-level problem, not a missing component. Compute, networking, model lifecycle, physical state, and mission control must be designed together. Terrestrial AI systems provide abstractions for clusters, accelerators, schedulers, and serving; space systems provide predictable orbits, constrained links, and tightly managed platforms. The challenge is to connect these traditions without importing datacentre assumptions. We identify five problems that stand between isolated onboard demonstrations and space-native infrastructure.

6.1 Space-Native Infrastructure Abstractions

A first challenge is the absence of a shared abstraction. Terrestrial AI systems reason about servers, racks, clusters, jobs, services, and regions. Space systems are still usually built around one spacecraft, payload, mission, or pipeline. A model that runs on one satellite therefore does not automatically become portable, schedulable, or maintainable across another satellite, orbital plane, ground station, or mission profile.

The orbital unit is not simply a server. A satellite combines compute, storage, sensors, radios, attitude control, power generation, batteries, thermal paths, fault-management logic, and mission priorities. Its capacity varies with contacts, sunlight, eclipse, thermal exposure, and visibility. A constellation is not a static cluster either, because orbital planes, inter-satellite links, handovers, downlinks, and mission roles determine which nodes can cooperate. Useful abstractions must therefore expose orbit, contact, energy, thermal state, reliability role, storage pressure, and mission priority as scheduling resources.

The boundary is delicate. Exposing every spacecraft detail makes the platform unusable for AI developers; hiding physical state makes the runtime unsafe. A promising direction is a layered resource model in which satellites export forecasted envelopes for compute, power, thermal headroom, storage, and contacts, while the mission system retains authority over safety-critical rules. AI services would request objectives, such as latency, energy budget, degradation mode, and latest downlink time, and the infrastructure would map them to onboard execution, deferral, migration, or ground fallback. This requires resource descriptors, service-level objectives, telemetry schemas, and programming models that are both space-aware and usable by AI system builders.

6.2 Topology-Predictive Runtime and Cross-Layer Orchestration

Space networks are dynamic but not arbitrary. Visibility, contact duration, path length, and inter-satellite connectivity are largely driven by orbital mechanics and can be forecast. A runtime with this forecast can pre-position model artifacts, defer nonurgent movement, reserve downlink capacity, migrate service state before handover, and decide whether inference should run onboard, on a neighboring satellite, at a ground station, or in the cloud.

Topology decisions are entangled with compute and mission state. Model placement determines data movement; contacts determine whether checkpoints, adapters, or data products can synchronize; energy and thermal margins determine whether a visible node can accept work; and mission tasks may preempt AI services even when a path exists. A topology-aware runtime must therefore schedule communication together with compute, storage, power, and thermal state.

A central direction is to make contact prediction an operating-system-like primitive. The runtime should expose future topology, contact windows, expected link quality, and uncertainty to placement and admission control. Offline predictions can provide a baseline plan, online telemetry can correct it, and the orchestrator can revise placement, migration, and downlink decisions. Forecasts will still fail because of weather, pointing constraints, congestion, faults, emergency events, and demand bursts. Degradation modes, such as smaller models, batched inference, compressed features, skipped updates, or local-only execution, should therefore be explicit and schedulable.

6.3 Energy-Thermal-Aware Sustainable AI Serving

Onboard AI capacity is not a fixed operations-per-second number. It is a time-varying service envelope shaped by sunlight, eclipse, battery depth of discharge, thermal history, heat rejection, payload activity, and safety rules. A spacecraft may run a model briefly but not sustain it as a service. Section 5 shows this directly: thermal protection or energy limits can preempt computation before nominal accelerator capability is exhausted.

This changes AI serving. Datacentre serving systems often optimize throughput, latency, utilization, and cost under stable facility support. In orbit, serving must also preserve energy margin, thermal headroom, and mission safety. A request should be admitted only if the predicted physical trajectory remains safe, and the best placement may be the node whose future contact, battery state, radiator orientation, and mission schedule can support the service without later shutdown.

The research problem is to model service capacity as a forecasted physical resource. The runtime should estimate how many inferences can be served before eclipse, a thermal threshold, or a downlink opportunity; which model variant fits a power cap; how work should move among orbit, ground, and cloud as thermal margin changes; and how urgent mission events should be balanced against background updates or batch analytics. This requires telemetry-driven models of power draw, heat accumulation, cooldown, and workload-dependent accelerator behavior. It also requires safe degradation: smaller batches or models, lower frame rates, delayed tasks, compressed outputs, checkpointing, or transfer to another node or ground backend.

6.4 AI State, Model Lifecycle, and Distributed Execution

AI infrastructure manages more than tasks and packets. It manages weights, adapters, prompts, checkpoints, KV caches, intermediate features, sensor data, telemetry, logs, downlink queues, versions,

validation reports, and rollback states. In terrestrial clouds these artifacts rely on abundant storage, fast networks, and frequent operator intervention. In space they must survive intermittent links, limited storage, narrow uplinks, radiation faults, thermal interruption, and long lifetimes.

Lifecycle control is the first challenge. Models launched with the spacecraft will become stale as tasks, environments, and ground pipelines evolve. Updates are necessary but risky: they may exceed contact windows, conflict with dependencies, be unsafe under current power conditions, or be impossible to roll back. A space-native model registry should span ground and orbit, tracking versions, dependencies, validation, hardware compatibility, energy and memory profiles, and rollback plans. Deployment should be staged, contact-aware, mission-aware, and verified before activation.

Execution-state continuity is the second challenge. Stateful services carry context that is expensive to reconstruct, including KV caches, partial outputs, intermediate features, and runtime checkpoints. When a node approaches a thermal limit, loses contact, or enters a mission-critical period, the infrastructure must decide whether to checkpoint, migrate, discard, recompute, or degrade state. Distributed execution adds state placement: workloads may split across sensor-side filtering, feature extraction, onboard inference, ground refinement, and cloud aggregation, but split points are useful only if they fit the network and physical envelope. The long-term goal is a lifecycle system in which models and state can be updated, audited, recovered, and composed across space and ground.

6.5 Trustworthy Evaluation, Resilience, and Governance

The final challenge is trust. Space AI infrastructure operates inside non-maintainable, high-risk, long-lifetime physical systems. Accuracy, latency, throughput, and model size are necessary but insufficient. A service can be accurate in isolation yet unusable if it consumes too much energy, overheats the platform, misses contacts, cannot recover, or cannot be audited after autonomous action.

Benchmarks should therefore report model quality together with energy margin, thermal headroom, storage pressure, link use, recovery time, update cost, and mission value. They should include realistic orbital traces, contact plans, power and thermal profiles, workload bursts, and failure events. Digital twins can connect simulation, hardware-in-the-loop testing, and in-orbit validation, but only if calibrated with flight telemetry.

Resilience and governance must be built into the infrastructure layer. Radiation events, software faults, model drift, corrupted updates, cyber attacks, link disruption, and physical protection events need explicit handling through authenticated delivery, verified rollback, degraded modes, fault injection, checkpoint recovery, and audit trails. High-impact actions should use verify-before-act mechanisms that separate model recommendation from mission authority when ground supervision is delayed. APIs, logs, update mechanisms, evaluation metrics, and control boundaries should also reflect spectrum, orbital slots, ground-station sharing, cybersecurity, sensed-data privacy, debris risk, and responsible autonomy from the beginning.

7 Research Roadmap

The path toward AI infrastructure in space should be staged. Near term, the goal is not a full orbital datacenter but onboard AI that is measurable, updateable, and safe beyond isolated demonstrations. Mid term, satellites, ground stations, and cloud backends should form a hybrid infrastructure loop. Long term, constellations should expose compute, storage, networking, and physical-state resources as programmable services.

7.1 Near Term: AI-Enabled Satellites

In the near term, single-satellite AI demonstrations should become reproducible and maintainable systems. Missions should report not only whether a model ran, but also its energy cost, thermal trajectory, memory footprint, contact use, recovery behavior, and operational constraints. These traces and profiles create the empirical basis for schedulers and benchmarks. Near-term systems should also standardize the minimum lifecycle: package a model, validate it on the ground, deliver it through constrained links, activate it safely, monitor it in orbit, and roll it back if needed. Full autonomy is unnecessary. A hybrid-first design can keep heavy training, validation, and policy decisions on the ground while satellites execute bounded inference, filtering, and local adaptation. The success criterion is deployment, monitoring, update, and evaluation under realistic orbital constraints, not a one-time demonstration.

7.2 Mid Term: Hybrid Space-Ground AI Infrastructure

The mid-term stage should connect AI-enabled satellites into hybrid space-ground infrastructure. Satellites act as frontends close to sensors and users, while ground stations and cloud backends provide heavier computation, model management, long-term storage, and fleet-level control. The boundary expands from one spacecraft to the loop of satellite, inter-satellite link, gateway, mission control, model registry, and cloud service. The defining capability is predictive orchestration. Runtimes should use contact, energy, thermal, storage, and mission forecasts to decide where models and state reside. A model update may be staged before a contact window, a checkpoint moved before a thermal interruption, and a sensing workload run onboard during low-latency events but deferred when contacts make ground processing viable. Mid-term systems should support stateful AI services, contact-aware placement, energy-thermal-aware scheduling, multi-satellite collaboration, calibrated digital twins, and service-level objectives that include physical margins and recovery behavior.

7.3 Long Term: Orbital AI Infrastructure

In the long term, AI infrastructure in space could become a shared orbital computing substrate rather than a mission-specific feature. Constellations may expose compute-storage-network resources as programmable infrastructure, multiple services may share orbital nodes, and AI tasks may execute across spacecraft, ground stations, and terrestrial clouds as one distributed system. This requires constellation-level resource abstraction, bounded autonomy, distributed state management, service migration, secure multi-tenant

isolation, and governance-aware scheduling. Orbital contact, energy, thermal headroom, reliability state, and sustainability constraints must be native resources. The goal is not to move all AI computation into space, but to place capability where data, latency, resilience, or mission value justify it and coordinate that capability with terrestrial infrastructure. Progress should be evaluated by whether AI services become sustainable, recoverable, updateable, and composable across the space-ground continuum.

8 Conclusion

AI infrastructure in space is emerging as a new systems problem at the intersection of AI systems, networking, and spacecraft engineering. This paper has reviewed the computing foundations, examined the physical constraints that distinguish orbital platforms from terrestrial infrastructure, and used in-orbit case studies to illustrate how these constraints affect computing capacity, software operation, and service continuity. Together, these observations show that space-native AI infrastructure requires resource abstractions and system mechanisms that account for time-varying connectivity, energy, thermal conditions, and recoverability. Many challenges remain before such infrastructure can support reliable and scalable AI services. Progress will depend on tighter integration across computing, networking, system software, and spacecraft design, as well as realistic evaluation through simulation, testbeds, and in-orbit deployment. We hope this paper helps define a research agenda toward programmable, manageable, and dependable AI infrastructure in space.

References

- [1] Luiz André Barroso, Urs Hölzle, and Parthasarathy Ranganathan. *The Datacenter as a Computer: Designing Warehouse-Scale Machines*. Morgan & Claypool, 3rd edition, 2018.
- [2] Norman P. Jouppi et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pages 1–12, 2017.
- [3] Norman P. Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Cliff Young, Xiang Zhou, Zongwei Zhou, and David Patterson. TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023.
- [4] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, et al. MLPerf inference benchmark. In *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 446–459, 2020.
- [5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.
- [6] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems (EuroSys)*, pages 1–17, 2015.
- [7] Benjamin Hindman, Andy Konwinski, Matei Zaharia, Ali Ghodsi, Anthony D. Joseph, Randy Katz, Scott Shenker, and Ion Stoica. Mesos: A platform for fine-grained resource sharing in the data center. In *8th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 295–308, 2011.
- [8] Arjun Singh, Joon Ong, Amit Agarwal, Glen Anderson, Ashby Armistead, Roy Bannon, Seb Boving, Gaurav Desai, Bob Felderman, Paulie Germano, et al. Jupiter rising: A decade of clos topologies and centralized control in Google’s datacenter network. In *Proceedings of the ACM SIGCOMM 2015 Conference*, pages 183–197, 2015.
- [9] Leon Poutievski, Omid Mashayekhi, Joon Ong, Arjun Singh, Mukarram Tariq, Rui Wang, Jianan Zhang, Virginia Beauregard, Patrick Conner, Steve Gribble, et al. Jupiter evolving: Transforming Google’s datacenter network via optical

- circuit switches and software-defined networking. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 66–85, 2022.
- [10] Gianluca Giuffrida, Luca Fanucci, Gabriele Meoni, Marko Batić, Liam Buckley, Alan Dunne, Casper van Dijk, Marco Esposito, Johannes Hefele, Nele Ver-cruyssen, Gianluca Furano, Michele Pastena, and Josef Aschbacher. The ϕ -sat-1 mission: The first on-board deep neural network demonstrator for satellite earth observation. *IEEE Transactions on Geoscience and Remote Sensing*, 60(1):1–14, 2022.
 - [11] Bradley Denby and Brandon Lucia. Orbital edge computing: Nanosatellite constellations as a new class of computer system. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 939–954, 2020.
 - [12] Zhishu Shen et al. A survey of next-generation computing technologies in space-air-ground integrated networks. *ACM Computing Surveys*, 56(1):1–40, 2024.
 - [13] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 521–538, 2022.
 - [14] Jinfeng Wen, Jianshu Zhao, Zixi Zhu, Xiaomin Zhang, Qi Liang, Ao Zhou, and Shanguang Wang. SateLight: A satellite application update framework for satellite computing. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering*, 2025.
 - [15] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, et al. RDMA over ethernet for distributed training at Meta scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 57–70, 2024.
 - [16] Jinkyu Lee, Eugene Kim, and Kang G. Shin. Design and management of satellite power systems. In *Proceedings of the IEEE Real-Time Systems Symposium (RTSS)*, pages 97–106, 2013.
 - [17] Theodore D. Swanson and Gajanana C. Birur. NASA thermal control technologies for robotic spacecraft. *Applied Thermal Engineering*, 23(9):1055–1065, 2003.
 - [18] Sabrina Corpino, Matteo Caldera, Marco Masoero, Fabio Nichele, and Nicole Viola. Thermal design and analysis of a nanosatellite in low earth orbit. *Acta Astronautica*, 115:247–261, 2015.
 - [19] Paul E. Dodd and Lloyd W. Massengill. Basic mechanisms and modeling of single-event upset in digital microelectronics. *IEEE Transactions on Nuclear Science*, 50(3):583–602, 2003.
 - [20] James R. Schwank, Marty R. Shaneyfelt, Daniel M. Fleetwood, James A. Felix, Paul E. Dodd, Philippe Paillet, and Véronique Ferlet-Cavrois. Radiation effects in MOS oxides. *IEEE Transactions on Nuclear Science*, 55(4):1833–1853, 2008.
 - [21] Kevin Fall. A delay-tolerant network architecture for challenged internets. In *Proceedings of the ACM SIGCOMM Conference*, pages 27–34, 2003.
 - [22] Bradley Denby, Krishna Chintalapudi, Ranveer Chandra, Brandon Lucia, and Shadi Noghbi. Kodan: Addressing the computational bottleneck in space. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 392–403. ACM, 2023.
 - [23] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, Omega, and Kubernetes. *Communications of the ACM*, 59(5):50–57, 2016.
 - [24] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elilol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 561–577, 2018.
 - [25] Juan A. Fraire et al. Contact plan design for delay-tolerant space sensor networks. *Ad Hoc Networks*, 2021.
 - [26] European Space Agency. Φ sat: Artificial Intelligence for Earth Observation. ESA Web Page, 2020.
 - [27] Hewlett Packard Enterprise. HPE Spaceborne Computer-2 returns to the international space station. HPE News Release, January 2024.
 - [28] NASA Spinoff. Cutting-edge computing goes spaceborne. NASA Spinoff, February 2025.
 - [29] Microsoft Azure. Introducing Azure Orbital: Process satellite data at cloud scale. Microsoft Azure Blog, September 2020.
 - [30] Axiom Space. Axiom Space to launch orbital data center nodes to support national security, commercial, international customers. Axiom Space News Release, April 2025.
 - [31] Akash Sriram. SpaceX aims to launch orbital AI computing tests by end of next year, sources say. Reuters, June 2026.
 - [32] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. NVIDIA A100 tensor core GPU: Performance and innovation. *IEEE Micro*, 41(2):29–35, 2021.
 - [33] Peter Mattson, Christine Cheng, Gregory Diamos, Cody Coleman, Paulius Micikevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bittorf, et al. MLPerf training benchmark. In *Proceedings of Machine Learning and Systems (MLSys)*, 2020.
 - [34] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, et al. Alibaba HPN: A data center network for large language model training. In *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024.
 - [35] Muhammad Tirmazi, Adam Barker, Nan Deng, Md E. Haque, Zhijiang Gene Qin, Steven Hand, Mor Harchol-Balter, and John Wilkes. Borg: The next generation. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, pages 1–14, 2020.
 - [36] Luiz Barroso, Mike Marty, David Patterson, and Parthasarathy Ranganathan. Attack of the killer microseconds. *Communications of the ACM*, 60(4):48–54, 2017.
 - [37] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Brijesh Warrier, Nithish Mahalingam, and Ricardo Bianchini. Characterizing power management opportunities for LLMs in the cloud. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 207–222, 2024.
 - [38] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Brijesh Warrier, Nithish Mahalingam, and Ricardo Bianchini. POLCA: Power oversubscription in LLM cloud providers. *arXiv preprint arXiv:2308.12908*, 2023.
 - [39] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibao Nong, et al. MegaScale: Scaling large language model training to more than 10,000 GPUs. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 745–760, 2024.
 - [40] Qinghao Hu, Zhisheng Ye, Zerui Wang, Guoteng Wang, Meng Zhang, Qiaoling Chen, Peng Sun, Dahua Lin, Xiaolin Wang, Yingwei Luo, Yonggang Wen, and Tianwei Zhang. Characterization of large language model development in the datacenter. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 709–729, 2024.
 - [41] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
 - [42] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
 - [43] Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, Fan Yang, and Lidong Zhou. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 595–610, 2018.
 - [44] Juncheng Gu, Mosharaf Chowdhury, Kang G. Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Liu, and Chuanxiong Guo. Tiresias: A GPU cluster manager for distributed deep learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 485–500, 2019.
 - [45] Kshiteej Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Shuchi Chawla. Themis: Fair and efficient GPU cluster scheduling. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 289–304, 2020.
 - [46] Yanghua Peng, Yixin Bao, Yangrui Chen, Chuan Wu, and Chuanxiong Guo. Optimus: An efficient dynamic resource scheduler for deep learning clusters. In *Proceedings of the Thirteenth European Conference on Computer Systems (EuroSys)*, pages 1–14, 2018.
 - [47] Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R. Ganger, and Eric P. Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 1–18, 2021.
 - [48] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhemiaka, Amar Phanishayee, and Matei Zaharia. Heterogeneity-aware cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 481–498, 2020.
 - [49] Suhas Jayaram Subramanya, Daiyaan Arfeen, Shouxu Lin, Aurick Qiao, Zhihao Jia, and Gregory R. Ganger. Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 642–657, 2023.
 - [50] Pengfei Zheng, Rui Pan, Tarannum Khan, Shivaram Venkataraman, and Aditya Akella. Shockwave: Fair and efficient cluster scheduling for dynamic adaptation in machine learning. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 703–723, 2023.
 - [51] Diandian Gu, Yihao Zhao, Yinmin Zhong, Yifan Xiong, Zhenhua Han, Peng Cheng, Fan Yang, Gang Huang, Xin Jin, and Xuanzhe Liu. ElasticFlow: An elastic serverless training platform for distributed deep learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 266–280, 2023.
 - [52] Changho Hwang, Taehyun Kim, Sunghyun Kim, Jinwoo Shin, and Kyoungsoo Park. Elastic resource sharing for distributed deep learning. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 721–739, 2021.

- [53] Jiamin Li, Hong Xu, Yibo Zhu, Zherui Liu, Chuanxiong Guo, and Cong Wang. Lyra: Elastic scheduling for deep learning clusters. In *Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys)*, pages 835–850, 2023.
- [54] Hanyu Zhao, Zhenhua Han, Zhi Yang, Quanlu Zhang, Fan Yang, Lidong Zhou, Mao Yang, Francis C. M. Lau, Yuqi Wang, Yifan Xiong, and Bin Wang. HiveD: Sharing a GPU cluster for deep learning with guarantees. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 515–532, 2020.
- [55] Wencong Xiao, Shiru Ren, Yong Li, Yang Zhang, Pengyang Hou, Zhi Li, Yihui Feng, Wei Lin, and Yangqing Jia. AntMan: Dynamic scaling on GPU clusters for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 533–548, 2020.
- [56] Zhihao Bai, Zhen Zhang, Yibo Zhu, and Xin Jin. PipeSwitch: Fast pipelined context switching for deep learning applications. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 499–514, 2020.
- [57] Gangmuk Lim, Jeongseob Ahn, Wencong Xiao, Youngjin Kwon, and Myeongjae Jeon. Zico: Efficient GPU memory sharing for concurrent DNN training. In *2021 USENIX Annual Technical Conference (ATC)*, pages 161–175, 2021.
- [58] Jayashree Mohan, Amar Phanishayee, Janardhan Kulkarni, and Vijay Chidambaram. Looking beyond GPUs for DNN scheduling on multi-tenant clusters. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 579–596, 2022.
- [59] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. Analysis of large-scale multi-tenant GPU clusters for DNN training workloads. In *2019 USENIX Annual Technical Conference (ATC)*, pages 947–960, 2019.
- [60] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 945–960, 2022.
- [61] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [62] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashiunkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on GPU clusters using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.
- [63] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Xu Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [64] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, pages 1–15, 2019.
- [65] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. ZeRO: Memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2020.
- [66] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. Alpa: Automating inter- and intra-operator parallelism for distributed deep learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 559–578, 2022.
- [67] Zhihao Jia, Matei Zaharia, and Alex Aiken. Beyond data and model parallelism for deep neural networks. In *Proceedings of Machine Learning and Systems (MLSys)*, 2019.
- [68] Yuanzhong Xu, HyoukJoong Lee, Dehao Chen, Blake Hechtman, Yanping Huang, Rahul Joshi, Maxim Krikun, Dmitry Lepikhin, Andy Ly, Marcello Maggioni, Ruoming Pang, Noam Shazeer, Shibo Wang, Tao Wang, Yonghui Wu, and Zhifeng Chen. GSPMD: General and scalable parallelization for ML computation graphs. *arXiv preprint arXiv:2105.04663*, 2021.
- [69] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pages 3505–3506, 2020.
- [70] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damanika, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. PyTorch FSDP: Experiences on scaling fully sharded data parallel. *Proceedings of the VLDB Endowment*, 16(12):3848–3860, 2023.
- [71] Paul Barham, Aakanksha Chowdhery, Jeff Dean, Sanjay Ghemawat, Steven Hand, Daniel Hurt, Michael Isard, Hyeontaek Lim, Ruoming Pang, Sudip Roy, Brennan Saeta, Parker Schuh, Ryan Sepassi, Laurent El Shafey, Chandramohan A. Thekkath, and Yonghui Wu. Pathways: Asynchronous distributed dataflow for ML. In *Proceedings of Machine Learning and Systems (MLSys)*, 2022.
- [72] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021.
- [73] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- [74] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In *Proceedings of the 39th International Conference on Machine Learning (ICML)*, pages 18332–18346, 2022.
- [75] Jayashree Mohan, Amar Phanishayee, and Vijay Chidambaram. CheckFreq: Frequent, fine-grained DNN checkpointing. In *19th USENIX Conference on File and Storage Technologies (FAST)*, pages 203–216, 2021.
- [76] Assaf Eisenman, Kiran Kumar Matam, Steven Ingram, Dheevatsa Mudigere, Raghuraman Krishnamoorthi, Krishnakumar Nair, Misha Smelyanskiy, and Murali Annamaram. Check-N-Run: A checkpointing system for training deep learning recommendation models. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 929–943, 2022.
- [77] Zhuang Wang, Zhen Jia, Shuai Zheng, Zhen Zhang, Xinwei Fu, T. S. Eugene Ng, and Yida Wang. GEMINI: Fast failure recovery in distributed training with in-memory checkpoints. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 364–381, 2023.
- [78] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. Bamboo: Making preemptible instances resilient for affordable training of large DNNs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 497–513, 2023.
- [79] Insu Jang, Zhenning Yang, Zhen Zhang, Xin Jin, and Mosharaf Chowdhury. Oobleck: Resilient distributed training of large models using pipeline templates. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 382–395, 2023.
- [80] Sanjith Athlur, Nitika Saran, Muthian Sivathanu, Ramachandran Ramjee, and Nipun Kwatra. Varuna: Scalable, low-cost training of massive deep learning models. In *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys)*, pages 472–487, 2022.
- [81] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 193–210, 2024.
- [82] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 117–134, 2024.
- [83] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Inigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132, 2024.
- [84] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST)*, pages 155–170, 2025.
- [85] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 663–679, 2023.
- [86] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Lumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 173–191, 2024.
- [87] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. ServerlessLLM: Low-latency serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 135–153, 2024.
- [88] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [89] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.

- [90] Daniel Crankshaw, Xin Wang, Giulio Zhou, Michael J. Franklin, Joseph E. Gonzalez, and Ion Stoica. Clipper: A low-latency online prediction serving system. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 613–627, 2017.
- [91] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving DNNs like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 443–462, 2020.
- [92] Francisco Romero, Qian Li, Neeraja J. Yadwadkar, and Christos Kozyrakis. INFaaS: Automated model-less inference serving. In *2021 USENIX Annual Technical Conference (ATC)*, pages 397–411, 2021.
- [93] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 31094–31116, 2023.
- [94] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pages 19274–19286, 2023.
- [95] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2022.
- [96] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, He Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- [97] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [98] Inigo del Portillo, Bruce G. Cameron, and Edward F. Crawley. A technical comparison of three low earth orbit satellite constellation systems to provide global broadband. *Acta Astronautica*, 159:123–135, 2019.
- [99] Aizaz U. Chaudhry and Halim Yanikomeroglu. Laser intersatellite links in a Starlink constellation: A classification and analysis. *IEEE Vehicular Technology Magazine*, 16(2):48–56, 2021.
- [100] Deepak Vasishth and Ranveer Chandra. A distributed and hybrid ground station network for low earth orbit satellites. In *Proceedings of the 19th ACM Workshop on Hot Topics in Networks (HotNets '20)*, pages 190–196, 2020.
- [101] Debopam Bhattacharjee, Simon Kassing, Melissa Licciardello, and Ankit Singla. In-orbit computing: An outlandish thought experiment? In *Proceedings of the 19th ACM Workshop on Hot Topics in Networks (HotNets '20)*, pages 197–204, 2020.
- [102] Markus Werner. A dynamic routing concept for ATM-based satellite personal communication networks. *IEEE Journal on Selected Areas in Communications*, 15(8):1636–1648, 1997.
- [103] Eylem Ekici, Ian F. Akyildiz, and Michael D. Bender. A distributed routing algorithm for datagram traffic in LEO satellite networks. *IEEE/ACM Transactions on Networking*, 9(2):137–147, 2001.
- [104] Juan A. Fraire and Jorge M. Finochietto. Design challenges in contact plans for disruption-tolerant satellite networks. *IEEE Communications Magazine*, 53(5):163–169, 2015.
- [105] Dara Ron et al. Time-dependent network topology optimization for LEO satellite constellations. In *IEEE INFOCOM 2025 - IEEE Conference on Computer Communications*, 2025.
- [106] Debopam Bhattacharjee, Waqar Aqeel, Ilker Nadi Bozkurt, Anthony Aguirre, Balakrishnan Chandrasekaran, P. Brighten Godfrey, Gregory Laughlin, Bruce Maggs, and Ankit Singla. Gearing up for the 21st century space race. In *Proceedings of the 17th ACM Workshop on Hot Topics in Networks (HotNets '18)*, pages 113–119, 2018.
- [107] Debopam Bhattacharjee and Ankit Singla. Network topology design at 27,000 km/hour. In *Proceedings of the 15th International Conference on Emerging Networking Experiments and Technologies (CoNEXT '19)*, pages 341–354, 2019.
- [108] Mark Handley. Delay is not an option: Low latency routing in space. In *Proceedings of the 17th ACM Workshop on Hot Topics in Networks (HotNets '18)*, pages 85–91, 2018.
- [109] Mark Handley. Using ground relays for low-latency wide-area routing in megaconstellations. In *Proceedings of the 18th ACM Workshop on Hot Topics in Networks (HotNets '19)*, pages 125–132, 2019.
- [110] Yuanjie Li, Hewu Li, Wei Liu, Lixin Liu, Wei Zhao, Yimei Chen, Jianping Wu, Qian Wu, Jun Liu, Zeqi Lai, and Han Qiu. A networking perspective on Starlink’s self-driving LEO mega-constellation. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking (MobiCom '23)*, pages 1–16, 2023.
- [111] Scott Burleigh, Kevin Fall, and Edward J. Birrane. Bundle Protocol Version 7. RFC 9171, IETF, 2022.
- [112] Giuseppe Araniti, Nikolaos Bezirgiannidis, Edward Birrane, Igor Bisio, Scott Burleigh, Carlo Caini, Marius Feldmann, Mario Marchese, John Segui, and Kiyohisa Suzuki. Contact graph routing in DTN space networks: overview, enhancements and performance. *IEEE Communications Magazine*, 53(3):38–46, 2015.
- [113] Deepak Vasishth, Jayanth Shenoy, and Ranveer Chandra. L2D2: Low latency distributed downlink for LEO satellites. In *Proceedings of the 2021 ACM SIGCOMM Conference*, pages 151–164, 2021.
- [114] Bill Tao, Maleeha Masood, Indranil Gupta, and Deepak Vasishth. Transmitting, fast and slow: Scheduling satellite traffic through space and time. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking (MobiCom '23)*, pages 1–15, 2023.
- [115] Xuyang Cao and Xinyu Zhang. SaTCP: Link-layer informed TCP adaptation for highly dynamic LEO satellite networks. In *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*, pages 1–10, 2023.
- [116] François Michel, Martino Trevisan, Danilo Giordano, and Olivier Bonaventure. A first look at Starlink performance. In *Proceedings of the 22nd ACM Internet Measurement Conference (IMC '22)*, pages 130–136, 2022.
- [117] Nitinder Mohan, Andrew E. Ferguson, Hendrik Cech, Rohan Bose, Prakita Rayyan Renatin, Mahesh K. Marina, and Jörg Ott. A multifaceted look at Starlink performance. In *Proceedings of the ACM Web Conference 2024 (WWW '24)*, 2024.
- [118] Simon Kassing, Debopam Bhattacharjee, André Baptista Águas, Jens Eirik Saethre, and Ankit Singla. Exploring the “internet from space” with Hypatia. In *Proceedings of the 20th ACM Internet Measurement Conference (IMC '20)*, pages 214–229, 2020.
- [119] Zeqi Lai, Hewu Li, and Jihao Li. StarPerf: Characterizing network performance for emerging mega-constellations. In *2020 IEEE 28th International Conference on Network Protocols (ICNP)*, pages 1–11, 2020.
- [120] Zeqi Lai, Hewu Li, Yangtao Deng, Qian Wu, Jun Liu, Yuanjie Li, Jihao Li, Lixin Liu, Weisen Liu, and Jianping Wu. StarryNet: Empowering researchers to evaluate futuristic integrated space and terrestrial networks. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23)*, pages 1309–1324, 2023.
- [121] Tobias Pfandzelter and David Bermbach. Celestial: Virtual software system testbeds for the LEO edge. In *Proceedings of the 23rd ACM/IFIP International Middleware Conference (Middleware '22)*, pages 69–81, 2022.
- [122] Wenhao Lu, Zhiyuan Wang, Hefan Zhang, Shan Zhang, and Hongbin Luo. OpenSN: An open source library for emulating LEO satellite networks. *IEEE Transactions on Parallel and Distributed Systems*, 36(8):1574–1590, 2025.
- [123] Suvar Basak, Amitangshu Pal, and Debopam Bhattacharjee. LEOcraft: Towards designing performant LEO networks. In *2025 USENIX Annual Technical Conference (USENIX ATC '25)*, pages 789–813, 2025.
- [124] Liying Wang, Qing Li, Yuhan Zhou, Zhao Feng Luo, Donghao Zhang, Shangguang Wang, Xuanzhe Liu, and Chenren Xu. Emulating space computing networks with RHONE. In *2025 USENIX Annual Technical Conference (USENIX ATC '25)*, pages 815–831, 2025.
- [125] Gianluca Giuffrida, Lorenzo Diana, Francesco de Gioia, Gionata Benelli, Gabriele Meoni, Massimiliano Donati, and Luca Fanucci. Cloudscout: A deep neural network for on-board cloud detection on hyperspectral images. *Remote Sensing*, 12(14):2205, 2020.
- [126] Georges Labrèche, David Evans, Tom Mladenov, Dominik Marszk, Vasundhara Shiradhonkar, and Vladimir Zelenevskiy. Artificial intelligence for autonomous planning and scheduling of image acquisition with the smartcam app on-board the ops-sat spacecraft. In *AIAA SCITECH 2022 Forum*, 2022.
- [127] Shreeyam Kacker, Alex Meredith, Kerri Cahoy, and Georges Labrèche. Machine learning image processing algorithms onboard ops-sat. In *Proceedings of the 36th Annual Small Satellite Conference*, 2022.
- [128] Gabriele Meoni, Marcus Mörtens, Dawa Derksen, Kenneth See, Toby Lighthouse, Anthony Sécher, Arnaud Martin, David Rijlaarsdam, Vincenzo Fanizza, and Dario Izzo. The ops-sat case: A data-centric competition for onboard satellite image classification. *Astrodynamics*, 8:507–528, 2024.
- [129] Bogdan Ruszczyk, Krzysztof Kotowski, David Evans, and Jakub Nalepa. The ops-sat benchmark for detecting anomalies in satellite telemetry. *Scientific Data*, 12(1):710, 2025.
- [130] Dennis D. Langer, Milica Orlandić, Sivert Bakken, Roger Birkeland, Joseph L. Garrett, Tor A. Johansen, and Asgeir J. Sørensen. Robust and reconfigurable on-board processing for a hyperspectral imaging small satellite. *Remote Sensing*, 15(15):3756, 2023.
- [131] Jon A. Justo, Joseph L. Garrett, Dennis D. Langer, Marie Bøe Henriksen, Radu Tudor Ionescu, and Tor Arne Johansen. An open hyperspectral dataset with sea-land-cloud ground-truth from the hypso-1 satellite. In *2023 13th Workshop on Hyperspectral Imaging and Signal Processing: Evolution in Remote Sensing (WHISPERS)*, pages 1–5, 2023.
- [132] Jon Alvarez Justo, Dennis D. Langer, Simen Berg, Jens Niekte, Radu Tudor Ionescu, Per Gunnar Kjeldsberg, and Tor Arne Johansen. Hyperspectral image segmentation for optimal satellite operations: In-orbit deployment of 1d-cnn. *Remote Sensing*, 17(4):642, 2025.
- [133] Beatriz Soret, Israel Leyva-Mayorga, Antonio M. Mercado-Martínez, Marco Moretti, Antonio Jurado-Navas, Marc Martínez-Gost, Celia Sánchez de Miguel,

- Ainoa Salas-Prendes, and Petar Popovski. Semantic and goal-oriented edge computing for satellite earth observation. *arXiv preprint arXiv:2408.15639*, 2024.
- [134] Zhenjiang Zhang, Wenyu Zhang, and Fan-Hsun Tseng. Satellite mobile edge computing: Improving qos of high-speed satellite-terrestrial networks using edge computing techniques. *IEEE Network*, 33(1):70–76, 2019.
- [135] Tobias Pfandzelter and David Bermbach. Qos-aware resource placement for leo satellite edge computing. In *Proceedings of the 2022 IEEE 6th International Conference on Fog and Edge Computing (ICFEC)*, pages 66–72. IEEE, 2022.
- [136] Fábio Diniz Rossi, Paulo Silas Severo de Souza, and Marcelo Caggiani Luizelli. Resource allocation on low-earth orbit edge infrastructure: Taxonomy, survey, and research challenges. *IEEE Access*, 13:40992–41010, 2025.
- [137] Jiaqi Cao, Shengli Zhang, Qingxia Chen, Houtian Wang, Mingzhe Wang, and Naijin Liu. Computing-aware routing for leo satellite networks: A transmission and computation integration approach. *IEEE Transactions on Vehicular Technology*, 72(12):16607–16623, 2023.
- [138] Bo Wang, Tong Feng, and Dongyan Huang. A joint computation offloading and resource allocation strategy for leo satellite edge computing system. In *Proceedings of the 2020 IEEE 20th International Conference on Communication Technology (ICCT)*, pages 649–655. IEEE, 2020.
- [139] Zhengyu Song, Yuanquan Hao, Yuanwei Liu, and Xin Sun. Energy-efficient multiaccess edge computing for terrestrial-satellite internet of things. *IEEE Internet of Things Journal*, 8(18):14202–14218, 2021.
- [140] Qingqing Tang, Zesong Fei, Bin Li, and Zhu Han. Computation offloading in leo satellite networks with hybrid cloud and edge computing. *IEEE Internet of Things Journal*, 8(11):9164–9176, 2021.
- [141] Yunbo Qu, Tingting Zhang, Yuan Feng, Tongyang Xu, and Zijian Guo. Computation offloading and resource allocation for e2e tasks in satellite edge computing networks. *Space: Science & Technology*, 4:0144, 2024.
- [142] Bill Tao, Om Chhabra, Ishani Janveja, Indranil Gupta, and Deepak Vasisht. Known knowns and unknowns: Near-realtime earth observation via query bifurcation in serval. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 809–824, 2024.
- [143] Zhengqian Xu, Peiying Zhang, Chengcheng Li, Hailong Zhu, Guanjuan Xu, and Chenhua Sun. A collaborative inference algorithm in low-earth-orbit satellite network for unmanned aerial vehicle. *Drones*, 7(9):575, 2023.
- [144] Jinglong Guan, Qiyang Zhang, Ilir Murturi, Praveen Kumar Donta, Schahram Dustdar, and Shangguang Wang. Collaborative inference in dnn-based satellite systems with dynamic task streams. In *ICC 2024 – IEEE International Conference on Communications*, pages 3803–3808. IEEE, 2024.
- [145] Meng Zhang, Hongjian Shi, and Ruhui Ma. Edge-distributed iot services assist the economic sustainability of leo satellite constellation construction. *Sustainability*, 16(4):1599, 2024.
- [146] Lingling Wu and Jingjing Zhang. Fedgsm: Efficient federated learning for leo constellations with gradient staleness mitigation. In *2023 IEEE 24th International Workshop on Signal Processing Advances in Wireless Communications (SPAWC)*, pages 356–360. IEEE, 2023.
- [147] Yuanming Shi, Li Zeng, Jingyang Zhu, Yong Zhou, Chunxiao Jiang, and Khaled B. Letaief. Satellite federated edge learning: Architecture design and convergence analysis. *IEEE Transactions on Wireless Communications*, 23(10):15212–15229, 2024.
- [148] Yuanming Shi, Jingyang Zhu, Chunxiao Jiang, Linling Kuang, and Khaled Ben Letaief. Satellite edge artificial intelligence with large models: Architectures and technologies. *Science China Information Sciences*, 68(7):170302, 2025.
- [149] Ruolin Xing, Mengwei Xu, Ao Zhou, Qing Li, Yiran Zhang, Feng Qian, and Shangguang Wang. Deciphering the enigma of satellite computing with COTS devices: Measurement and analysis. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*, pages 420–435, 2024.
- [150] Pablo A. Anigstein and Ricardo S. Sánchez-Peña. Analysis of solar panel orientation in low altitude satellites. *IEEE Transactions on Aerospace and Electronic Systems*, 34(2):569–578, 1998.
- [151] Aman Kansal, Jason Hsu, Sadaf Zahedi, and Mani B. Srivastava. Power management in energy harvesting sensor networks. *ACM Transactions on Embedded Computing Systems*, 6(4):32:1–32:38, 2007.
- [152] Josiah Hester, Kevin Storer, and Jacob Sorber. Timely execution on intermittently powered batteryless sensors. In *Proceedings of the 15th ACM Conference on Embedded Network Sensor Systems (SenSys)*, pages 17:1–17:13, 2017.
- [153] Jiong Luo and Niraj K. Jha. Battery-aware static scheduling for distributed real-time embedded systems. In *Proceedings of the Design Automation Conference (DAC)*, pages 444–449, 2001.
- [154] Daler Rakhmatov and Sarma Vrudhula. Energy management for battery-powered embedded systems. *ACM Transactions on Embedded Computing Systems*, 2(3):277–324, 2003.
- [155] Eugene Kim, Jinkyu Lee, Liang He, Youngmoon Lee, and Kang G. Shin. Offline guarantee and online management of power demand and supply in cyber-physical systems. In *Proceedings of the IEEE Real-Time Systems Symposium (RTSS)*, pages 89–98, 2016.
- [156] Alexei Colin, Emily Ruppel, and Brandon Lucia. A reconfigurable energy storage architecture for energy-harvesting devices. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 767–781, 2018.
- [157] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. EIE: Efficient inference engine on compressed deep neural network. In *Proceedings of the 43rd Annual International Symposium on Computer Architecture (ISCA)*, pages 243–254, 2016.
- [158] Yu-Hsin Chen, Tushar Krishna, Joel Emer, and Vivienne Sze. Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE Journal of Solid-State Circuits*, 52(1):127–138, 2017.
- [159] Arun Raghavan, Yixin Luo, Anuj Chandawalla, Marios Papaefthymiou, Kevin P. Pipe, Thomas F. Wenisch, and Milo M. K. Martin. Computational sprinting. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 1–12, 2012.
- [160] Qing Li, Shangguang Wang, Chenren Xu, Xiao Ma, Mengwei Xu, Ao Zhou, Ruolin Xing, Boyuan Yang, Zuo Zhu, Ying Zhang, and Xuanzhe Liu. Exploring real-time satellite computing: From energy and thermal perspectives. In *Proceedings of the IEEE Real-Time Systems Symposium (RTSS)*, pages 161–173, 2024.
- [161] Qinqin Tang, Renchao Xie, Zeru Fang, Tao Huang, Tianjiao Chen, Ran Zhang, and F. Richard Yu. Joint service deployment and task scheduling for satellite edge computing: A two-timescale hierarchical approach. *IEEE Journal on Selected Areas in Communications*, 42(5):1063–1079, 2024.
- [162] Jiacheng Liu, Xiaozhi Zhu, Tongqiao Xu, Xiaofeng Hou, and Chao Li. Spaceexit: Enabling efficient adaptive computing in space with early exits. In *Proceedings of the USENIX Annual Technical Conference (USENIX ATC)*, pages 1343–1358, 2025.
- [163] Graham Gobieski, Brandon Lucia, and Nathan Beckmann. Intelligence beyond the edge: Inference on intermittent embedded systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.
- [164] Bashima Islam and Shahrar Nirjon. Zygarde: Time-sensitive on-device deep inference and adaptation on intermittently-powered systems. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 4(3):82:1–82:29, 2020.
- [165] C. Figus, L. Ounougha, P. Bonzom, W. Supper, and C. Puillet. Capillary fluid loop developments in astrium. *Applied Thermal Engineering*, 23(9):1085–1098, 2003.
- [166] Christine Hoa, Benoît Demolder, and Alain Alexandre. Roadmap for developing heat pipes for ALCATEL SPACE’s satellites. *Applied Thermal Engineering*, 23(9):1099–1108, 2003.
- [167] David Brooks and Margaret Martonosi. Dynamic thermal management for high-performance microprocessors. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 171–182, 2001.
- [168] Kevin Skadron, Mircea R. Stan, Wei Huang, Sivakumar Velusamy, Karthik Sankaranarayanan, and David Tarjan. Temperature-aware microarchitecture. In *Proceedings of the 30th Annual International Symposium on Computer Architecture (ISCA)*, pages 2–13, 2003.
- [169] Wei Huang, Mircea R. Stan, Kevin Skadron, Karthik Sankaranarayanan, Shougata Ghosh, and Sivakumar Velusamy. Hotspot: A compact thermal modeling methodology for early-stage VLSI design. *IEEE Transactions on Very Large Scale Integration Systems*, 14(5):501–513, 2006.
- [170] James Donald and Margaret Martonosi. Techniques for multicore thermal management: Classification and new exploration. In *Proceedings of the 33rd Annual International Symposium on Computer Architecture (ISCA)*, pages 78–88, 2006.
- [171] Murat Bulut and Nedim Sözbir. Analytical investigation of a nanosatellite panel surface temperatures for different altitudes and panel combinations. *Applied Thermal Engineering*, 75:1076–1083, 2015.
- [172] Róbert Kovács and Viktor Józsa. Thermal analysis of the SMOG-1 pocketcube satellite. *Applied Thermal Engineering*, 139:506–513, 2018.
- [173] Mario Bonnici, Pierluigi Mollicone, Maurizio Fenech, and Marc Anthony Azopardi. Analytical and numerical models for thermal related design of a new pico-satellite. *Applied Thermal Engineering*, 159:113908, 2019.
- [174] A. Torres, D. Mishkinis, and T. Kaya. Mathematical modeling of a new satellite thermal architecture system connecting the east and west radiator panels and flight performance prediction. *Applied Thermal Engineering*, 65(1–2):623–632, 2014.
- [175] N. D. Anh, N. N. Hieu, P. N. Chung, and N. T. Anh. Thermal radiation analysis for small satellites with single-node model using techniques of equivalent linearization. *Applied Thermal Engineering*, 94:607–614, 2016.
- [176] E. Escobar, M. Díaz, and J. C. Zagal. Evolutionary design of a satellite thermal control system: Real experiments for a CubeSat mission. *Applied Thermal Engineering*, 105:490–500, 2016.
- [177] Ahmed Elhefnawy, Ali Elmaihy, and Ahmed Elweteedy. Passive thermal control design and analysis of a university-class satellite. *Journal of Thermal Analysis*

- and Calorimetry, 147:13633–13651, 2022.
- [178] Hadi Esmaeilzadeh, Emily Blem, Renée St. Amant, Karthikeyan Sankaralingam, and Doug Burger. Dark silicon and the end of multicore scaling. In *Proceedings of the 38th Annual International Symposium on Computer Architecture (ISCA)*, pages 365–376, 2011.
 - [179] Mohamed A. Goma, Michael D. Powell, and T. N. Vijaykumar. Heat-and-run: Leveraging SMT and CMP to manage power density through the operating system. In *Proceedings of the 11th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 260–270, 2004.
 - [180] Kiran Puttaswamy and Gabriel H. Loh. Thermal herding: Microarchitecture techniques for controlling hotspots in high-performance 3d-integrated processors. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 193–204, 2007.
 - [181] Mohamed M. Sabry, Ayse K. Coskun, David Atienza, Tajana Simunic Rosing, and Thomas Brunschwiler. Energy-efficient multiobjective thermal control for liquid-cooled 3-d stacked architectures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 30(12):1883–1896, 2011.
 - [182] Ayse Kivildim Coskun, Tajana Simunic Rosing, and Kenny C. Gross. Proactive temperature balancing for low cost thermal management in MPSoCs. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 250–257, 2008.
 - [183] Francesco Zanini, Mohamed M. Sabry, David Atienza, and Giovanni De Micheli. Hierarchical thermal management policy for high-performance 3d systems with liquid cooling. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 1(2):88–101, 2011.
 - [184] Chao Wang, Xiao Ma, Chuanxiu Chi, Ao Zhou, Ruolin Xing, and Shangguang Wang. Temperature- and energy-aware dynamic task scheduling and computing resource allocation for satellite computing. *IEEE Transactions on Mobile Computing*, 25(5):6245–6260, 2026.
 - [185] Shubhendu S. Mukherjee, Joel S. Emer, and Steven K. Reinhardt. The soft error problem: An architectural perspective. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 243–247, 2005.
 - [186] Véronique Ferlet-Cavrois, Lloyd W. Massengill, and Pascale Gouker. Single event transients in digital CMOS—a review. *IEEE Transactions on Nuclear Science*, 60(3):1767–1790, 2013.
 - [187] Paolo Rech. Artificial neural networks for space and safety-critical applications: Reliability issues and potential solutions. *IEEE Transactions on Nuclear Science*, 71(4):377–404, 2024.
 - [188] F. Libano, B. Wilson, Michael Wirthlin, Paolo Rech, and Jennifer Brunhaver. Understanding the impact of quantization, accuracy, and radiation on the reliability of convolutional neural networks on FPGAs. *IEEE Transactions on Nuclear Science*, 67(7):1478–1484, 2020.
 - [189] José M. Badia, Germán León, José A. Belloch, Mario García-Valderas, Almudena Lindoso, and Luis Entrena. Comparison of parallel implementation strategies in GPU-accelerated system-on-chip under proton irradiation. *IEEE Transactions on Nuclear Science*, 69(3):444–452, 2022.
 - [190] H. B. Wang, Y. S. Wang, J. H. Xiao, S. L. Wang, and T. J. Liang. Impact of single-event upsets on convolutional neural networks in xilinx zynq FPGAs. *IEEE Transactions on Nuclear Science*, 68:394–401, 2021.
 - [191] Xabier Iturbe, Balaji Venu, Emre Ozer, Jean-Luc Poupat, Grégoire Gimenez, and Hans-Ulrich Zurek. The arm triple core lock-step (TCLS) processor. *ACM Transactions on Computer Systems*, 36(3):7:1–7:30, 2019.
 - [192] Andrew M. Keller and Michael J. Wirthlin. Benefits of complementary SEU mitigation for the LEON3 soft processor on SRAM-based FPGAs. *IEEE Transactions on Nuclear Science*, 64(1):519–528, 2017.
 - [193] Matthew J. Marinella. Radiation effects in advanced and emerging nonvolatile memories. *IEEE Transactions on Nuclear Science*, 68(5):546–572, 2021.
 - [194] Michael Rogenmoser, Yvan Tortorella, Davide Rossi, Francesco Conti, and Luca Benini. Hybrid modular redundancy: Exploring modular redundancy approaches in RISC-V multi-core computing clusters for reliable processing in space. *ACM Transactions on Cyber-Physical Systems*, 9(1):8:1–8:29, 2025.
 - [195] Brendan Cully, Geoffrey Lefebvre, Dutch T. Meyer, Mike Feeley, Norman C. Hutchinson, and Andrew Warfield. Remus: High availability via asynchronous virtual machine replication. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pages 161–174, 2008.
 - [196] George Candea, Shinichi Kawamoto, Yuichi Fujiki, Greg Friedman, and Armando Fox. Microreboot—a technique for cheap recovery. In *Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 31–44, 2004.
 - [197] Michael Le and Yuval Tamir. Rehype: Enabling VM survival across hypervisor failures. In *Proceedings of the ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE)*, pages 63–74, 2011.
 - [198] Jason Ansel, Kapil Arya, and Gene Cooperman. DMTCP: Transparent checkpointing for cluster computations and the desktop. In *Proceedings of the IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 1–12, 2009.
 - [199] Zitao Chen, Niranjana Narayanan, Bo Fang, Guanpeng Li, Karthik Pattabiraman, and Nathan DeBardeleben. Tensorfi: A flexible fault injection framework for tensorflow applications. In *Proceedings of the IEEE International Symposium on Software Reliability Engineering (ISSRE)*, pages 426–435, 2020.
 - [200] Abhishek Tyagi, Yiming Gan, Shaoshan Liu, Bo Yu, Paul N. Whatmough, and Yuhao Zhu. Thales: Formulating and estimating architectural vulnerability factors for DNN accelerators. In *Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023.
 - [201] Thomas C. Bressoud and Fred B. Schneider. Hypervisor-based fault tolerance. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, pages 1–11, 1995.
 - [202] N. Asokan, Thomas Nyman, Norrathep Rattanavipanon, Ahmad-Reza Sadeghi, and Gene Tsudik. ASSURED: Architecture for secure software update of realistic embedded devices. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 37(11):2290–2300, 2018.
 - [203] Zitao Chen, Guanpeng Li, and Karthik Pattabiraman. A low-cost fault corrector for deep neural networks through range restriction. In *Proceedings of the IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 1–13, 2021.
 - [204] Shuguang Feng, Shantanu Gupta, Amin Ansari, and Scott Mahlke. Shoestring: Probabilistic soft error reliability on the cheap. In *Proceedings of the 15th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 385–396, 2010.
 - [205] Eric Cheng, Shahrzad Mirkhani, Lukasz G. Szafaryn, Chen-Yong Cher, Hyung-min Cho, Kevin Skadron, Mircea R. Stan, Klas Lilja, Jacob A. Abraham, Pradip Bose, and Subhasish Mitra. CLEAR: Cross-layer exploration for architecting resilience—combining hardware and software techniques to tolerate soft errors in processor cores. In *Proceedings of the Design Automation Conference (DAC)*, pages 68:1–68:6, 2016.
 - [206] Zhuo Cheng, Bradley Denby, Kyle McCreary, and Brandon Lucia. Eagleeye: Nanosatellite constellation design for high-coverage, high-resolution sensing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 117–132, 2024.
 - [207] Zhuo Cheng and Brandon Lucia. Geoduck: Nanosatellite constellation scheduling for low latency event detection. In *Proceedings of the ACM Conference on Embedded Networked Sensor Systems (SenSys)*, pages 547–559, 2025.
 - [208] Khan M. F. Shahil and Alexander A. Balandin. Graphene—multilayer graphene nanocomposites as highly efficient thermal interface materials. *Nano Letters*, 12(2):861–867, 2012.
 - [209] H. Ji, D. P. Sellan, M. T. Pettes, X. Kong, J. Ji, L. Shi, and R. S. Ruoff. Enhanced thermal conductivity of phase change materials with ultrathin-graphite foams for thermal energy storage. *Energy & Environmental Science*, 7:1185–1192, 2014.
 - [210] Marco Durante and Francis A. Cucinotta. Physical basis of radiation protection in space travel. *Reviews of Modern Physics*, 83:1245–1281, 2011.
 - [211] Roberto Pastore, Andrea Delfini, M. Albano, Antonio Vricella, M. Marchetti, F. Santoni, and F. Piergentili. Outgassing effect in polymeric composites exposed to space environment thermal-vacuum conditions. *Acta Astronautica*, 170:466–471, 2020.
 - [212] Sandeep Kumar Samal, Shreepad Panth, Kambiz Samadi, Mehdi Saedi, Yang Du, and Sung Kyu Lim. Fast and accurate thermal modeling and optimization for monolithic 3d ICs. In *Proceedings of the Design Automation Conference (DAC)*, pages 206:1–206:6, 2014.
 - [213] Gengjie Chen, Jian Kuang, Zhiliang Zeng, Hang Zhang, Evangeline F. Y. Young, and Bei Yu. Minimizing thermal gradient and pumping power in 3d IC liquid cooling network design. In *Proceedings of the Design Automation Conference (DAC)*, pages 70:1–70:6, 2017.
 - [214] Guglielmo S. Aglietti, Marcello Remedia, Matteo Appolloni, and Andrew Kiley. On the spacecraft structure model validation and test philosophy. *ALAA Journal*, 57(5):2109–2122, 2019.
 - [215] Davide Pederbelli, Pietro Nali, and Adriano Calvi. Euclid spacecraft sine vibration test prediction through the virtual shaker testing technique. *Journal of Spacecraft and Rockets*, 60(5):1517–1526, 2023.
 - [216] Randy Spicer, Tom Cote, Nguyen Do, and Jonathan Black. Design, analysis, test, and model correlation of a flight-like additively manufactured ESPA-class satellite. *Journal of Spacecraft and Rockets*, 2022.
 - [217] Abdullah Al-Yafawi, Da Yu, Seungbae Park, James Pitarresi, and Soonwan Chung. Reliability assessment of electronic components under random vibration loading. In *Proceedings of the IEEE Electronic Components and Technology Conference (ECTC)*, 2009.
 - [218] John T. Hwang, Dae Young Lee, James W. Cutler, and Joaquim R. R. A. Martins. Large-scale multidisciplinary optimization of a small satellite’s design and operation. *Journal of Spacecraft and Rockets*, 51(5):1648–1663, 2014.
 - [219] Ali Ravanbakhsh and Sebastian Franchini. Multiobjective optimization applied to structural sizing of low cost university-class microsatellite projects. *Acta Astronautica*, 79:212–220, 2012.
 - [220] Justin S. Gray, John T. Hwang, Joaquim R. R. A. Martins, Kenneth T. Moore, and Bret A. Naylor. Openmdao: An open-source framework for multidisciplinary

- design, analysis, and optimization. *Structural and Multidisciplinary Optimization*, 2019.
- [221] Runxi Wang, Ziheng Wang, Ting Lin, Jacob M. Raby, Mircea R. Stan, and Xinfei Guo. Cool-3d: An end-to-end thermal-aware framework for early-phase design space exploration of microfluidic-cooled 3DICs. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 15(4):659–673, 2025.
- [222] Joaquim R. R. A. Martins and Andrew B. Lambe. Multidisciplinary design optimization: A survey of architectures. *AIAA Journal*, 51(9):2049–2075, 2013.